

EBOOK 2026

166

Skills para Claude Code

O guia definitivo para dominar a CLI
da Anthropic e multiplicar sua produtividade.

16 capítulos · 166 skills · prompts prontos para usar

>_ We{Stack}

Sumario

Intro Introducao

Cap. 01 Code Generation

Cap. 02 Debugging

Cap. 03 Refactoring

Cap. 04 Testing

Cap. 05 Git Version Control

Cap. 06 Documentation

Cap. 07 Api Development

Cap. 08 Database

Cap. 09 Devops

Cap. 10 Security

Cap. 11 Performance

Cap. 12 Frontend

Cap. 13 Backend

Cap. 14 Ai MI

Cap. 15 Project Management

Cap. 16 Code Review

Bonus Workflows Avancados

Introducao

Bem-vindo ao guia mais completo de Claude Code do mercado.

O que é o Claude Code?

Claude Code é a CLI (Command Line Interface) oficial da Anthropic que roda diretamente no seu terminal. Diferente de chatbots web, o Claude Code:

- **Le seu projeto inteiro** — entende a estrutura de pastas, dependências, configurações e código existente
- **Executa comandos** — cria arquivos, instala dependências, roda testes, faz commits
- **Opera no contexto real** — não trabalha com código hipotético, mas com o seu projeto de verdade
- **Suporta qualquer linguagem** — TypeScript, Python, Go, Rust, Java e qualquer outra

Como instalar

```
# Instale via npm (requer Node.js 18+)
npm install -g @anthropic-ai/claude-code

# Verifique a instalacao
claude --version

# Inicie em qualquer projeto
cd seu-projeto
claude
```

Como este ebook esta organizado

O ebook contém **166 skills** organizadas em **16 capitulos** por area de atuacao:

Cap.	Area	Skills
01	Code Generation & Scaffolding	14
02	Debugging & Error Fixing	12
03	Refactoring & Optimization	12
04	Testing & QA	12
05	Git & Version Control	10
06	Documentation	12
07	API Development	12
08	Database	10
09	DevOps & Deploy	10
10	Security	10
11	Performance	10
12	Frontend	10
13	Backend	10
14	AI / ML	8
15	Project Management	8
16	Code Review	6
Bonus	Workflows Avancados	—

Formato de cada skill

Cada skill segue este formato padronizado:

```
### Skill #NNN - Titulo da Skill
**Categoria:** area | **Nivel:** Basico/Intermediario/Avancado

**Quando usar:** Situacao especifica em que esta skill e util.

**Prompt:**
  0 prompt exato para colar no Claude Code.

**Output esperado:**
  - 0 que o Claude Code vai entregar

**Dica pro:** Dica avancada para extrair o maximo.
```

Como usar: Copie o prompt, cole no terminal onde o Claude Code está rodando e adapte os detalhes entre colchetes [assim] para o seu contexto.

Dicas gerais para melhores resultados

1. **Seja específico** — "Crie uma API REST com Express, TypeScript, Zod e Prisma" > "Crie uma API"

2. **De contexto** — O Claude Code lê seu projeto, mas mencionar frameworks e padrões que você usa ajuda
3. **Itere** — Use a primeira resposta como base e refine com follow-ups
4. **Use CLAUDE.md** — Configure um arquivo CLAUDE.md na raiz do projeto com instruções permanentes
5. **Combine skills** — As skills mais poderosas são combinações (veja o capítulo Bonus)

Configuração básica recomendada

Crie um arquivo `CLAUDE.md` na raiz do seu projeto:

```
# Instruções para o Claude Code

## Stack
- Runtime: Node.js 20 + TypeScript 5
- Framework: Next.js 14 (App Router)
- Banco: PostgreSQL + Prisma ORM
- Testes: Vitest + Playwright
- Estilo: Tailwind CSS

## Convenções
- Use TypeScript strict mode
- Siga o padrão de pastas existente
- Escreva testes para toda função pública
- Commits em conventional commits (feat:, fix:, chore:)
- Código e comentários em inglês, docs em português
```

Isso garante que toda skill que você executar vai seguir os padrões do seu projeto automaticamente.

Pronto? Vamos começar com a Skill #001.

Capítulo 1 — Code Generation & Scaffolding

14 skills para gerar projetos, componentes e estruturas completas do zero.

Skill #001 — Gerar Projeto Completo a Partir de Descrição

Categoria: Code Generation | **Nível:** Básico

Quando usar: Quando precisar criar um projeto inteiro do zero com uma única instrução.

Prompt:

```
Crie um projeto completo de [tipo do projeto] usando [stack]. Inclua: estrutura de pastas organizada, configuracao de TypeScript, linter (ESLint + Prettier), arquivo de ambiente (.env.example), README com instrucoes de setup, scripts de build/dev/test no package.json e um exemplo funcional de hello world com a stack rodando.
```

Output esperado:

- Estrutura de pastas completa
- Configurações de TS, ESLint, Prettier
- package.json com scripts úteis
- .env.example e .gitignore
- README com instruções
- Exemplo funcional rodando

Dica pro: Quanto mais específica a descrição da stack, melhor o resultado. Mencione versões quando possível.

Skill #002 — Scaffold de API REST com CRUD

Categoria: Code Generation | **Nível:** Básico

Quando usar: Quando precisar de uma API REST funcional com operações CRUD rapidamente.

Prompt:

```
Crie uma API REST completa em Node.js + Express + TypeScript para gerenciar [recurso]. Implemente CRUD completo (Create, Read, ReadById, Update, Delete) com: validacao de input usando Zod, error handling centralizado com codigos HTTP corretos, tipagem completa, separacao em camadas (routes, controllers, services, repositories) e testes unitarios com Vitest para cada endpoint.
```

Output esperado:

- Rotas CRUD completas
- Validação com Zod

- Error handling centralizado
- Camadas separadas (route/controller/service/repo)
- Testes unitarios
- Tipagem TypeScript completa

Dica pro: Substitua [recurso] pelo nome real (ex: "usuários", "produtos") para obter campos e validações mais realistas.

Skill #003 — Gerar Interfaces TypeScript a Partir de JSON

Categoria: Code Generation | **Nível:** Básico

Quando usar: Quando tiver um JSON de resposta de API e precisar dos tipos TypeScript.

Prompt:

Analise este JSON e gere interfaces TypeScript completas. Infira os tipos mais especificos possiveis (use union types para enums, Date para datas, branded types para IDs). Gere tambem type guards para validacao em runtime e um schema Zod equivalente. JSON: [cole o JSON aqui]

Output esperado:

- Interfaces TypeScript com tipos específicos
- Union types para campos enum
- Type guards para validação runtime
- Schema Zod equivalente
- Tipos exportados e organizados

Dica pro: Cole vários exemplos do mesmo JSON para que o Claude infira tipos opcionais e union types com mais precisao.

Skill #004 — Criar Componente React com Props Tipadas

Categoria: Code Generation | **Nível:** Básico

Quando usar: Quando precisar de um componente React completo com tipagem, estados e testes.

Prompt:

Crie um componente React chamado [NomeComponente] que [descricao da funcionalidade]. Inclua: interface de props completa com TSDoc, estados internos necessarios, tratamento de loading/error/empty states, acessibilidade (aria labels, keyboard navigation), estilizacao com [Tailwind/CSS Modules], memo/callback onde necessario para performance e teste unitario com Testing Library.

Output esperado:

- Componente React funcional tipado
- Interface de props com documentação

- Loading, error e empty states
- Acessibilidade implementada
- Estilizacao aplicada
- Teste unitario completo

Dica pro: Descreva o comportamento visual esperado — "card com imagem a esquerda, título e descrição a direita, botao de acao no rodape".

Skill #005 — Gerar Arquivos de Migration de Banco

Categoria: Code Generation | **Nível:** Intermediario

Quando usar: Quando precisar criar ou alterar tabelas no banco de dados.

Prompt:

```
Gere migrations para [descreva as mudancas no banco]. Use [Prisma/Knex/TypeORM/Drizzle] e inclua: migration up e down (reversivel), indexes para campos frequentemente consultados, constraints (unique, not null, foreign keys, check), valores default onde aplicavel e um seed file com dados de exemplo realistas para desenvolvimento.
```

Output esperado:

- Migration up (criar/alterar)
- Migration down (reverter)
- Indexes otimizados
- Constraints corretas
- Seed com dados realistas
- Schema atualizado

Dica pro: Sempre descreva as relações entre tabelas (1:N, N:N) para obter foreign keys e join tables corretas.

Skill #006 — Scaffold de Monorepo com Workspaces

Categoria: Code Generation | **Nível:** Intermediario

Quando usar: Quando precisar organizar múltiplos pacotes/apps em um único repositório.

Prompt:

```
Crie um monorepo com [pnpm workspaces/Turborepo/Nx] contendo: app web (Next.js), app mobile (React Native) ou API, pacote de UI components compartilhado, pacote de utils/helpers compartilhado, pacote de tipos/schemas compartilhado. Configure: build pipeline com cache, linting e testes globais, scripts de dev que rodam tudo junto, tsconfig compartilhado com paths e CI que roda apenas pacotes afetados por mudancas.
```

Output esperado:

- Monorepo estruturado com workspaces

- Apps e pacotes compartilhados
- Build pipeline com cache
- TSConfig compartilhado
- Scripts de dev integrados
- CI otimizado por pacote

Dica pro: Use Turborepo para projetos menores e Nx para projetos enterprise com muitos pacotes.

Skill #007 — Gerar CLI Tool com Parsing de Argumentos

Categoria: Code Generation | **Nível:** Intermediario

Quando usar: Quando precisar criar uma ferramenta de linha de comando.

Prompt:

```
Crie uma CLI tool em Node.js + TypeScript chamada [nome] que [descricao]. Use commander ou yargs para parsing de argumentos. Inclua: subcommands logicos, flags e opcoes com valores default, validacao de input, output colorido (chalk), spinner para operacoes longas (ora), arquivo de configuracao (.rc ou .config.js), help text detalhado e publicacao via npm (package.json configurado com bin).
```

Output esperado:

- CLI funcional com subcommands
- Parsing de argumentos configurado
- Output colorido e formatado
- Spinners para operações longas
- Arquivo de configuração
- Help text automático
- Pronta para publicar no npm

Dica pro: Adicione --json flag para output em JSON — facilita integração com outras ferramentas e pipelines.

Skill #008 — Criar Docker Compose Multi-Service

Categoria: Code Generation | **Nível:** Intermediario

Quando usar: Quando precisar orquestrar múltiplos serviços localmente com Docker.

Prompt:

```
Crie um docker-compose.yml completo para este projeto com todos os servicos necessarios. Inclua: app com hot-reload (volumes mapeados), banco de dados [PostgreSQL/MySQL/MongoDB] com volume persistente, Redis para cache/sessions, servico de email para dev (Mailpit), proxy reverso (Nginx ou Traefik), health checks em todos os servicos, networks separadas (frontend/backend), .env com variaveis configuraveis e Makefile com comandos uteis (make up, make down, make logs, make seed).
```

Output esperado:

- docker-compose.yml completo
- Todos os serviços com health checks
- Volumes persistentes
- Networks organizadas
- .env configurável
- Makefile com comandos úteis
- Documentação de uso

Dica pro: Use profiles do Docker Compose para separar serviços de dev (mailpit, adminer) dos de produção.

Skill #009 — Gerar Schema GraphQL + Resolvers

Categoria: Code Generation | **Nível:** Intermediario

Quando usar: Quando precisar criar uma API GraphQL completa.

Prompt:

```
Crie uma API GraphQL completa para [dominio] usando Apollo Server + TypeScript. Inclua: schema com tipos, queries, mutations e subscriptions, resolvers com data loaders para evitar N+1, autenticao via context, validacao de input, paginacao (cursor-based), error handling tipado, geracao de tipos com GraphQL Code Generator e testes dos resolvers.
```

Output esperado:

- Schema GraphQL completo
- Resolvers com DataLoaders
- Autenticação no context
- Paginação cursor-based
- Error handling tipado
- Code Generator configurado
- Testes de resolvers

Dica pro: Use DataLoaders desde o início — adicionar depois é muito mais trabalhoso e causa N+1 silencioso.

Skill #010 — Scaffold de Páginas Next.js (SSR/SSG)

Categoria: Code Generation | **Nível:** Intermediario

Quando usar: Quando precisar criar páginas Next.js com estratégias de renderizacao otimizadas.

Prompt:

```
Crie as paginas deste projeto Next.js (App Router) com a estrategia de renderizacao correta para cada uma. Para paginas estaticas use generateStaticParams, para dinamicas use SSR com cache revalidacao, para interativas use client components. Inclua: metadata SEO por pagina, loading.tsx e error.tsx, layout compartilhado, parallel routes onde aplicavel e not-found.tsx customizado. Siga a estrutura de pastas do App Router.
```

Output esperado:

- Páginas com estratégia de renderização correta
- Metadata SEO por página
- Loading e error states
- Layouts compartilhados
- Parallel routes (se aplicável)
- not-found customizado

Dica pro: Use 'use client' apenas nos componentes que realmente precisam de interatividade — não na página inteira.

Skill #011 — Gerar Schema Prisma a Partir de Linguagem Natural

Categoria: Code Generation | **Nível:** Básico

Quando usar: Quando precisar modelar seu banco de dados descrevendo as entidades em português.

Prompt:

```
Gere um schema Prisma completo para: [descreva seu dominio em linguagem natural, ex: "um sistema de e-commerce com usuarios, produtos com categorias, carrinho de compras, pedidos com itens e status de entrega, pagamentos e avaliacoes de produtos"]. Inclua: relacoes corretas (1:1, 1:N, N:N), campos de auditoria (createdAt, updatedAt), soft delete onde aplicavel, indexes para queries frequentes, enums para status e tipos, e validacoes no nivel do schema.
```

Output esperado:

- Schema Prisma completo
- Relações modeladas corretamente
- Campos de auditoria
- Indexes otimizados
- Enums para status
- Pronto para gerar migration

Dica pro: Descreva as regras de negocio junto com as entidades: "um usuário pode ter vários endereços mas apenas um principal".

Skill #012 — Criar package.json com Scripts Otimizados

Categoria: Code Generation | **Nível:** Básico

Quando usar: Quando precisar configurar scripts npm de forma profissional.

Prompt:

```
Analise este projeto e crie/otimize o package.json com scripts completos. Inclua: dev (com hot reload), build (otimizado para producao), start (producao), test (unitarios), test:e2e, test:coverage, lint (ESLint), lint:fix, format (Prettier), type-check (tsc --noEmit), db:migrate, db:seed, db:studio, docker:up, docker:down e pre-commit hook com husky + lint-staged. Organize dependencias vs devDependencies corretamente.
```

Output esperado:

- Scripts completos e organizados
- Husky + lint-staged configurados
- Dev, build, test, lint, format
- Scripts de banco de dados
- Scripts de Docker
- Dependências organizadas

Dica pro: Use concurrently para rodar scripts em paralelo no dev (ex: API + frontend + worker).

Skill #013 — Gerar .env.example a Partir do Codebase

Categoria: Code Generation | **Nível:** Básico

Quando usar: Quando precisar documentar todas as variaveis de ambiente usadas no projeto.

Prompt:

```
Analise todo o codebase e identifique todas as variaveis de ambiente utilizadas (process.env.*, import.meta.env.*, env()). Gere um arquivo .env.example com: todas as variaveis encontradas, valores de exemplo seguros (nao reais), comentarios explicando o proposito de cada variavel, agrupamento por categoria (Database, Auth, APIs, Email, etc.) e indicacao de quais sao obrigatorias vs opcionais. Crie tambem uma validacao que checa na inicializacao se todas as obrigatorias estao definidas.
```

Output esperado:

- .env.example completo e documentado
- Variaveis agrupadas por categoria
- Valores de exemplo seguros
- Indicacao obrigatória/opcional
- Script de validação na inicializacao

Dica pro: Adicione o .env.example ao git e o .env ao .gitignore — novos devs copiam e preenchem.

Skill #014 — Scaffold de Microservice com Health Check e Graceful Shutdown

Categoria: Code Generation | **Nível:** Avançado

Quando usar: Quando precisar criar um microservice production-ready desde o início.

Prompt:

Crie um microservice production-ready em [Node.js/Python/Go] que [descricao]. Inclua desde o inicio: health check endpoint (/health, /ready), graceful shutdown (SIGTERM/SIGINT handlers, drain connections), logging estruturado (JSON com correlation ID), metricas Prometheus (/metrics), configuracao por ambiente (12-factor app), Dockerfile multi-stage otimizado, testes e documentacao de API. O servico deve estar pronto para rodar em Kubernetes.

Output esperado:

- Microservice funcional
- Health check e readiness probe
- Graceful shutdown implementado
- Logging estruturado com correlation ID
- Métricas Prometheus
- Dockerfile otimizado
- Configuração por ambiente
- Testes e documentação

Dica pro: Implemente o graceful shutdown ANTES de qualquer lógica de negocio — adicionar depois é muito mais difícil.

Capítulo 2 — Debugging & Error Fixing

12 skills para diagnosticar e corrigir bugs de forma sistemática.

Skill #015 — Diagnosticar Erros de Runtime a Partir de Stack Traces

Categoria: Debugging | **Nível:** Básico

Quando usar: Quando sua aplicação lança um erro e você tem o stack trace.

Prompt:

Analise este stack trace e diagnostique o problema. Identifique: a causa raiz (não apenas o sintoma), o arquivo e linha exatos onde o erro origina, o fluxo de execução que levou ao erro e a correção mais segura. Aplique o fix e adicione um teste que reproduz o cenário do erro para prevenir regressão. Stack trace: [cole o stack trace]

Output esperado:

- Diagnóstico da causa raiz
- Localização exata do erro
- Fluxo de execução explicado
- Fix aplicado
- Teste de regressão criado

Dica pro: Cole o stack trace completo, incluindo "Caused by" — erros encadeados revelam a causa raiz real.

Skill #016 — Corrigir Erros de Tipo TypeScript

Categoria: Debugging | **Nível:** Intermediário

Quando usar: Quando TypeScript acusa erros de tipo que você não consegue resolver.

Prompt:

Analise e corrija todos os erros de tipo TypeScript neste projeto. Para cada erro: explique por que o tipo está incorreto, aplique a correção mais type-safe (evite any/as), se necessário crie types utilitários (Pick, Omit, Partial) ou type guards. Não use @ts-ignore ou type assertions inseguras. Rode tsc --noEmit para verificar que todos os erros foram resolvidos.

Output esperado:

- Todos os erros TS corrigidos
- Explicação de cada correção
- Type guards onde necessário

- Sem any, @ts-ignore ou type assertions inseguras
- tsc --noEmit passando

Dica pro: Se um tipo é muito complexo para inferir, crie um tipo nominal/branded — não use any.

Skill #017 — Debugar Race Conditions em Async/Await

Categoria: Debugging | **Nível:** Avançado

Quando usar: Quando sua aplicação tem comportamento intermitente causado por operações assíncronas.

Prompt:

Analise este código assíncrono e identifique race conditions. Procure por: Promise.all sem tratamento individual de erros, estado compartilhado entre operações concorrentes, falta de locks/mutex em recursos compartilhados, operações que deveriam ser sequenciais mas estão em paralelo, cleanup de recursos em caso de falha parcial. Corrija cada race condition e adicione testes que reproduzem o cenário concorrente.

Output esperado:

- Race conditions identificadas
- Correções aplicadas (locks, sequenciamento, error handling)
- Cleanup de recursos em falha parcial
- Testes de concorrência
- Explicação de cada correção

Dica pro: Use AbortController para cancelar operações pendentes quando o resultado já não é necessário.

Skill #018 — Identificar e Corrigir Memory Leaks

Categoria: Debugging | **Nível:** Avançado

Quando usar: Quando sua aplicação está consumindo cada vez mais memória ao longo do tempo.

Prompt:

Analise este projeto e identifique potenciais memory leaks. Procure por: event listeners não removidos, setInterval/setTimeout sem cleanup, closures que retêm referências grandes, caches sem limite de tamanho, referências circulares, streams não fechadas e subscriptions não canceladas. Para cada leak encontrado, aplique o fix e adicione cleanup no lifecycle correto (unmount, destroy, close).

Output esperado:

- Lista de memory leaks encontrados
- Event listeners com cleanup
- Timers com clearInterval/clearTimeout
- Caches com limite de tamanho (LRU)

- Streams fechadas corretamente
- Subscriptions canceladas
- Testes de cleanup

Dica pro: Em React, todo `useEffect` com `subscribe/addEventListener` DEVE retornar uma função de `cleanup`.

Skill #019 — Corrigir Problemas de CORS

Categoria: Debugging | **Nível:** Básico

Quando usar: Quando sua aplicação frontend não consegue acessar a API por erros de CORS.

Prompt:

```
Diagnostique e corrija os problemas de CORS neste projeto. Configure: origens permitidas por ambiente (dev: localhost, staging: *.staging.com, prod: dominio.com), metodos permitidos, headers permitidos (incluindo Authorization e Content-Type), credentials se necessario (cookies/auth headers), preflight caching (Access-Control-Max-Age), e tratamento especial para WebSockets. Nao use wildcard (*) em producao. Teste com requests de diferentes origens.
```

Output esperado:

- CORS configurado por ambiente
- Origens específicas (sem wildcard em prod)
- Métodos e headers corretos
- Credentials configurado
- Preflight caching
- Testes de CORS

Dica pro: CORS com credentials (cookies) NUNCA funciona com `origin: "*" — sempre especifique a origem exata.`

Skill #020 — Debugar Problemas de Layout CSS

Categoria: Debugging | **Nível:** Básico

Quando usar: Quando elementos visuais não estão posicionados ou dimensionados como esperado.

Prompt:

```
Analise os problemas de layout CSS neste projeto. Identifique e corrija: overflow causando scroll horizontal, elementos sobrepostos incorretamente, flex/grid items com tamanho errado, z-index wars, layout shifts (CLS) durante carregamento, problemas de responsividade em mobile (< 375px), textos cortados ou overflow e posicionamento absolute/fixed quebrado. Use as melhores praticas: min-width 0 em flex items, aspect-ratio para proporcoes, clamp() para tipografia responsiva.
```

Output esperado:

- Problemas de layout identificados
- Scroll horizontal eliminado

- Sobreposicoes corrigidas
- Responsividade testada
- CLS eliminado
- Boas práticas CSS aplicadas

Dica pro: Use outline: 1px solid red em elementos suspeitos para visualizar limites — e mais rápido que DevTools.

Skill #021 — Corrigir Conflitos de Versão de Dependências

Categoria: Debugging | **Nível:** Intermediario

Quando usar: Quando dependências conflitam entre si causando erros de build ou runtime.

Prompt:

```
Analise as dependencias deste projeto e resolva todos os conflitos. Identifique: versoes incompativeis entre pacotes, dependencias duplicadas em versoes diferentes, peer dependencies nao satisfeitos, vulnerabilidades conhecidas (npm audit), dependencias desatualizadas com breaking changes e packages deprecated com sugestoes de alternativas. Atualize de forma segura e rode os testes apos cada mudanca.
```

Output esperado:

- Conflitos de versão resolvidos
- Peer dependencies satisfeitos
- Vulnerabilidades corrigidas
- Dependências atualizadas
- Testes passando após mudanças
- Lock file atualizado

Dica pro: Atualize uma dependência por vez e rode testes entre cada uma — facilita identificar qual update quebrou algo.

Skill #022 — Diagnosticar e Corrigir Problemas N+1

Categoria: Debugging | **Nível:** Intermediario

Quando usar: Quando queries ao banco estao excessivamente lentas por problema de N+1.

Prompt:

```
Analise todas as queries deste projeto e identifique problemas N+1. Para cada caso: mostre a query N+1 (1 query para lista + N queries para detalhes), refatore para eager loading (JOIN/include), verifique se o ORM suporta batch loading, adicione DataLoader para contextos GraphQL se aplicavel e crie um teste que verifica o numero de queries executadas (query counter). Mostre o antes/depois com contagem de queries.
```

Output esperado:

- Queries N+1 identificadas

- Refatoracao para eager loading
- Contagem de queries antes/depois
- DataLoader (se GraphQL)
- Testes de contagem de queries

Dica pro: Adicione um middleware que loga o número de queries por request — N+1 se esconde atras de respostas "rapidas" em dev com poucos dados.

Skill #023 — Debugar Falhas de Conexão WebSocket

Categoria: Debugging | **Nível:** Intermediario

Quando usar: Quando conexões WebSocket caem, não conectam ou perdem mensagens.

Prompt:

Diagnostique e corrija problemas de WebSocket neste projeto. Verifique: handshake falhando (CORS, auth, upgrade headers), reconexao automatica com exponential backoff, heartbeat/ping-pong para detectar conexoes mortas, buffer de mensagens durante reconexao, tratamento de conexao em background tab (visibilitychange), limpeza de conexoes zombie no servidor e logging de eventos de conexao para debug.

Output esperado:

- Handshake corrigido
- Reconexao com exponential backoff
- Heartbeat implementado
- Buffer de mensagens offline
- Cleanup de conexões zombie
- Logging de eventos
- Tratamento de background tab

Dica pro: Implemente heartbeat a cada 30s — sem ele, conexões mortas ficam abertas e consomem recursos do servidor.

Skill #024 — Corrigir Bugs de Autenticação/Autorização

Categoria: Debugging | **Nível:** Intermediario

Quando usar: Quando usuários não conseguem logar, tokens expiram inesperadamente ou permissões falham.

Prompt:

Diagnostique e corrija todos os problemas de autenticao e autorizacao neste projeto. Verifique: fluxo de login (credenciais, OAuth), geracao e validacao de JWT (secret, expiracao, payload), refresh token flow (rotation, revogacao), middleware de auth (ordem no pipeline, rotas protegidas vs publicas), controle de permissoes (RBAC) e session management. Garanta que nao ha bypass de autenticao em nenhuma rota protegida.

Output esperado:

- Fluxo de login corrigido
- JWT configurado corretamente
- Refresh token flow funcionando
- Middleware de auth verificado
- Permissões checadas
- Nenhum bypass de auth
- Testes de auth completos

Dica pro: Crie um teste que acessa cada rota protegida sem token e verifica que retorna 401 — garante que nenhuma rota ficou desprotegida.

Skill #025 — Debugar Falhas no Build Pipeline

Categoria: Debugging | **Nível:** Intermediario

Quando usar: Quando o build falha no CI mas funciona localmente.

Prompt:

```
Diagnostique por que o build falha no CI/CD mas funciona local. Verifique: versoes de Node.js/npm/pnpm diferentes, dependencias nao commitadas no lock file, variaveis de ambiente faltando no CI, cache de dependencias obsoleto, ordem de execucao dos steps, permissoes de arquivos/diretorios, timeout de steps lentos e diferencas de OS (Linux no CI vs Mac/Windows local). Corrija e adicione validacoes no CI que previnam recorrência.
```

Output esperado:

- Causa da diferença CI vs local identificada
- Fix aplicado
- Versões fixadas no CI
- Variaveis de ambiente documentadas
- Cache de CI otimizado
- Válidações preventivas adicionadas

Dica pro: Adicione "engines" no package.json para fixar versões de Node/npm — previne builds com versões diferentes.

Skill #026 — Corrigir Bugs Específicos de Ambiente (Dev vs Prod)

Categoria: Debugging | **Nível:** Intermediario

Quando usar: Quando algo funciona em dev mas quebra em produção (ou vice-versa).

Prompt:

Diagnostique o bug que ocorre apenas em [producao/staging] mas nao em [dev]. Verifique: variaveis de ambiente diferentes, CORS/ dominio, HTTPS vs HTTP, cookies (SameSite, Secure, domain), minificacao/uglification quebrando codigo, polyfills faltando em producao, APIs externas com URLs diferentes por ambiente, banco de dados com dados/schema diferentes e feature flags. Identifique a causa e crie um teste que reproduz o cenario.

Output esperado:

- Causa do bug específico de ambiente
- Diferenças entre ambientes documentadas
- Fix aplicado
- Teste que reproduz o cenário
- Checklist de verificação pre-deploy

Dica pro: Mantenha um arquivo de diferenças entre ambientes — facilita diagnóstico quando o próximo bug ambiente-específico aparecer.

Capítulo 3 — Refactoring & Optimization

12 skills para melhorar a qualidade, legibilidade e manutenção do código existente.

Skill #027 — Extrair Funções Reutilizáveis de Código Repetido

Categoria: Refactoring | **Nível:** Básico

Quando usar: Quando identificar blocos de código duplicados ou muito similares no projeto.

Prompt:

Analise este projeto e identifique código duplicado ou muito similar. Para cada duplicação: extraia uma função reutilizável com nome descritivo, parametrize as diferenças, adicione tipagem completa, coloque no módulo/arquivo mais adequado e substitua todas as ocorrências pela função extraída. Garanta que os testes existentes continuam passando.

Output esperado:

- Duplicações identificadas
- Funções reutilizáveis extraídas
- Ocorrências substituídas
- Tipagem completa
- Testes passando

Dica pro: Se a duplicação envolve mais de 3 ocorrências, vale a extração. Para 2, avalie caso a caso.

Skill #028 — Converter Callbacks para Async/Await

Categoria: Refactoring | **Nível:** Básico

Quando usar: Quando o código usa callbacks aninhados (callback hell) que dificultam leitura.

Prompt:

Refatore todo o código com callbacks neste projeto para async/await. Para cada conversão: substitua callbacks por Promises (promisify se necessário), use async/await em vez de .then().catch(), trate erros com try/catch, mantenha paralelismo onde aplicável (Promise.all), adicione tipagem de retorno e garanta que o comportamento permanece idêntico. Rode os testes após cada conversão.

Output esperado:

- Callbacks convertidos para async/await
- Error handling com try/catch
- Paralelismo mantido com Promise.all
- Tipagem de retorno adicionada

- Testes passando

Dica pro: Use `util.promisify()` para converter APIs Node.js baseadas em callback (fs, crypto) automaticamente.

Skill #029 — Refatorar Class Components para Hooks (React)

Categoria: Refactoring | **Nível:** Intermediário

Quando usar: Quando tiver class components React que precisam ser modernizados.

Prompt:

```
Refatore todos os class components React deste projeto para functional components com hooks. Para cada componente:
converta state para useState, lifecycle methods para useEffect (com cleanup), refs para useRef, context para useContext,
extraia logica complexa em custom hooks reutilizaveis, mantenha a mesma API de props e garanta que o comportamento visual
e funcional e identico.
```

Output esperado:

- Class components convertidos para funcional
- State com useState
- Lifecycle com useEffect e cleanup
- Custom hooks extraídos
- API de props mantida
- Comportamento identico

Dica pro: Converta um componente por vez e rode os testes — conversoes em lote mascaram bugs introduzidos.

Skill #030 — Aplicar Principios SOLID ao Código Existente

Categoria: Refactoring | **Nível:** Avançado

Quando usar: Quando o código viola principios SOLID e esta difícil de manter ou estender.

Prompt:

```
Analise este projeto e identifique violacoes dos principios SOLID. Para cada violacao encontrada: identifique qual
principio esta sendo violado (S, O, L, I, D), explique o problema pratico que isso causa, refatore o codigo para aderir
ao principio e mostre como a mudanca facilita extensao futura. Foque nas violacoes que causam dor real (classes com
muitas responsabilidades, dependencias concretas que impedem testes), nao em purismo teorico.
```

Output esperado:

- Violacoes SOLID identificadas
- Explicacao do impacto práctico
- Refatoracao aplicada
- Antes/depois de cada mudança

- Testes atualizados

Dica pro: Comece pelo S (Single Responsibility) — e o que mais causa dor é o mais fácil de refatorar.

Skill #031 — Converter JavaScript para TypeScript Incrementalmente

Categoria: Refactoring | **Nível:** Intermediario

Quando usar: Quando quiser migrar um projeto JS para TS sem reescrever tudo de uma vez.

Prompt:

```
Configure e inicie a migracao incremental deste projeto JavaScript para TypeScript. Passo 1: configure tsconfig.json com allowJs e strict gradual. Passo 2: renomeie os arquivos mais criticos para .ts/.tsx (entry points, tipos compartilhados). Passo 3: adicione tipos nos arquivos renomeados (interfaces, type annotations). Passo 4: crie um script que mostra o progresso da migracao (% de arquivos TS). Nao tente migrar tudo de uma vez - priorize arquivos por impacto.
```

Output esperado:

- tsconfig.json configurado para migração
- Arquivos criticos renomeados para .ts
- Tipos adicionados nos arquivos migrados
- Script de progresso da migração
- Documentação do processo

Dica pro: Ative strict mode desde o início nos novos arquivos TS — migrar para strict depois é muito mais difícil.

Skill #032 — Decompor Funções Monolíticas em Unidades Compositivas

Categoria: Refactoring | **Nível:** Intermediario

Quando usar: Quando funções estão com 100+ linhas e fazem muitas coisas ao mesmo tempo.

Prompt:

```
Identifique funcoes com mais de 50 linhas neste projeto e decomponha-as. Para cada funcao longa: identifique as responsabilidades distintas, extraia cada uma em uma funcao pura com nome descritivo, mantenha a funcao original como orquestradora que chama as funcoes extraidas, adicione tipagem completa e garanta testabilidade de cada funcao isoladamente.
```

Output esperado:

- Funções longas identificadas
- Decomposição em funções menores
- Função orquestradora limpa
- Tipagem completa
- Testes unitarios por função
- Antes/depois em linhas de código

Dica pro: Funções puras (sem side effects) são mais fáceis de testar e reutilizar — priorize extrair lógica pura.

Skill #033 — Refatorar Lógica Condicional com Strategy Pattern

Categoria: Refactoring | **Nível:** Avançado

Quando usar: Quando if/else ou switch statements ficam enormes e difíceis de estender.

Prompt:

```
Identifique blocos condicionais complexos neste projeto (if/else chains com 4+ branches ou switch com 6+ cases) e refatore usando Strategy Pattern. Para cada caso: crie uma interface para a estratégia, implemente cada branch como uma classe/funcao que implementa a interface, use um registry/map para selecionar a estratégia correta e garanta que adicionar novas estratégias não requer modificar código existente (Open/Closed Principle).
```

Output esperado:

- Condicionais complexos identificados
- Interface de estratégia criada
- Estratégias implementadas
- Registry/map de seleção
- Extensível sem modificar existente
- Testes por estratégia

Dica pro: Nem todo if/else precisa de Strategy — use apenas quando o número de branches cresce com frequência.

Skill #034 — Normalizar Estruturas de Dados Aninhadas

Categoria: Refactoring | **Nível:** Intermediário

Quando usar: Quando dados profundamente aninhados dificultam acesso, atualização e performance.

Prompt:

```
Analise as estruturas de dados deste projeto e normalize dados profundamente aninhados. Crie: entities normalizadas com IDs como chaves (padrao Redux/normalizr), funcoes de normalizacao e denormalizacao, selectors para acessar dados relacionados e helpers para atualizar entidades sem mutacao. Mantenha compatibilidade com a API existente criando uma camada de transformacao.
```

Output esperado:

- Dados normalizados por entidade
- Funções de normalize/denormalize
- Selectors para acesso
- Helpers de atualização imutável
- Camada de transformação API

- Testes de transformação

Dica pro: Normalize apenas dados que são atualizados com frequência — dados read-only podem ficar aninhados.

Skill #035 — Extrair Lógica Compartilhada em Custom Hooks

Categoria: Refactoring | **Nível:** Intermediário

Quando usar: Quando lógica com estado e side effects se repete entre componentes React.

Prompt:

```
Analise os componentes React deste projeto e identifique logica repetida que pode ser extraida em custom hooks. Para cada hook extraido: nomeie com prefixo "use" descritivo, encapsule estado e side effects, exponha apenas a interface necessaria, adicione tipagem generica onde aplicavel e crie testes com renderHook. Exemplos comuns: useApi, useDebounce, useLocalStorage, useIntersectionObserver, useMediaQuery.
```

Output esperado:

- Lógica repetida identificada
- Custom hooks extraídos
- Interface limpa e tipada
- Generics onde aplicável
- Testes com renderHook
- Componentes simplificados

Dica pro: Um bom custom hook tem um único proposito e retorna [state, actions] — se retorna muita coisa, quebre em hooks menores.

Skill #036 — Simplificar Expressões Booleanas Complexas

Categoria: Refactoring | **Nível:** Básico

Quando usar: Quando condicionais estão com lógica booleana difícil de entender.

Prompt:

```
Identifique expressões booleanas complexas neste projeto (condições com 3+ operadores, negações duplas, nested ternaries) e simplifique. Para cada caso: extraia condições em variáveis com nomes descritivos (isEligible, hasPermission, canEdit), elimine negações duplas, substitua nested ternaries por early returns ou funções separadas e use De Morgan's laws para simplificar. O resultado deve ser legível sem precisar de comentários.
```

Output esperado:

- Expressões complexas identificadas
- Variáveis booleanas descritivas
- Negações duplas eliminadas
- Nested ternaries substituídos

- Lógica simplificada e legível

Dica pro: Se você precisa de um comentário para explicar uma condição, o nome da variável está ruim.

Skill #037 — Refatorar Camada de API com Repository Pattern

Categoria: Refactoring | **Nível:** Avançado

Quando usar: Quando chamadas de API estão espalhadas pelo código e difíceis de manter ou testar.

Prompt:

```
Refatore a camada de acesso a dados/API deste projeto usando o Repository Pattern. Crie: interface para cada repositório (UserRepository, ProductRepository), implementação concreta que faz as chamadas HTTP/DB reais, implementação fake/mock para testes, factory que retorna a implementação correta por ambiente, tipagem completa de request/response e error handling padronizado. Os componentes/services devem depender apenas da interface, nunca da implementação.
```

Output esperado:

- Interfaces de repositório
- Implementações concretas
- Implementações mock para testes
- Factory por ambiente
- Tipagem completa
- Error handling padronizado
- Dependency Injection configurado

Dica pro: Comece pelos repositórios mais usados — o ROI é imediato quando você pode trocar HTTP por mock em testes.

Skill #038 — Converter Loops Imperativos para Chains Funcionais

Categoria: Refactoring | **Nível:** Básico

Quando usar: Quando loops for/while fazem transformações que seriam mais claras com map/filter/reduce.

Prompt:

```
Identifique loops imperativos neste projeto que podem ser refatorados para métodos funcionais. Converta para: map() para transformações, filter() para filtragem, reduce() para acumulação, find()/some()/every() para buscas, flatMap() para arrays aninhados. Mantenha loops for quando performance crítica exigir (hot paths com 100k+ items). Adicione tipagem nos callbacks.
```

Output esperado:

- Loops imperativos identificados
- Convertidos para métodos funcionais
- Tipagem nos callbacks

- Performance mantida em hot paths
- Código mais legível e conciso

Dica pro: Prefira chains curtas (2-3 métodos). Se a chain fica longa, extraia em uma função nomeada.

Capítulo 4 — Testing & QA

12 skills para garantir qualidade com testes automatizados em todos os níveis.

Skill #039 — Gerar Testes Unitarios para Funções Existentes

Categoria: Testing | **Nível:** Básico

Quando usar: Quando funções existentes não tem testes e você precisa adicionar cobertura.

Prompt:

Analise todas as funcoes exportadas deste projeto e gere testes unitarios com [Vitest/Jest]. Para cada funcao: teste o caso feliz (happy path), casos edge (null, undefined, array vazio, string vazia, numeros negativos), tratamento de erros (input invalido, excecoes) e tipos de retorno. Use describe/it com nomes descritivos em portugues, arrange-act-assert pattern e factories para dados de teste. Mire cobertura minima de 80%.

Output esperado:

- Testes para todas as funções exportadas
- Happy path, edge cases e error handling
- Nomes descritivos em portugues
- Arrange-act-assert pattern
- Factories de dados de teste
- Cobertura 80%+

Dica pro: Foque em testar comportamento, não implementação — testes frágeis que quebram em qualquer refactor são piores que nenhum teste.

Skill #040 — Criar Suites de Testes de Integração

Categoria: Testing | **Nível:** Intermediario

Quando usar: Quando precisar testar interações entre módulos, API e banco de dados.

Prompt:

Crie testes de integracao para os fluxos criticos deste projeto. Use banco de dados real (container Docker via testcontainers ou banco de teste), teste cada endpoint da API com supertest, verifique: request → validacao → service -> repository → banco → response completo, setup/teardown de dados por teste (nao compartilhe estado entre testes), teste de transacoes e rollback e testes de autenticacao/autorizacao por rota.

Output esperado:

- Testes de integração por fluxo critico

- Banco de dados real (container)
- Request completo testado (end-to-end na API)
- Setup/teardown isolado
- Auth testada por rota
- Docker-compose para test environment

Dica pro: Use testcontainers para subir um banco descartavel por suite — elimina conflitos entre testes.

Skill #041 — Gerar Testes E2E com Playwright

Categoria: Testing | **Nível:** Intermediario

Quando usar: Quando precisar testar fluxos completos do usuário no browser.

Prompt:

```
Crie testes E2E com Playwright para os fluxos criticos do usuario neste projeto. Inclua: login/registro, navegacao principal, CRUD da feature principal, fluxo de compra/checkout (se aplicavel). Configure: Page Object Model para reutilizacao, fixtures para dados de teste, screenshots em falha, video recording, testes em 3 browsers (Chromium, Firefox, WebKit), mobile viewport e integracao com CI (GitHub Actions). Use locators acessiveis (getByRole, getByLabel).
```

Output esperado:

- Testes E2E dos fluxos criticos
- Page Object Model
- Fixtures configurados
- Screenshots e video em falha
- 3 browsers + mobile
- CI configurado
- Locators acessiveis

Dica pro: Use getByRole() e getByLabel() em vez de selectores CSS — são mais estaveis e também validam acessibilidade.

Skill #042 — Escrever Snapshot Tests para Componentes UI

Categoria: Testing | **Nível:** Básico

Quando usar: Quando precisar detectar mudanças visuais inesperadas em componentes.

Prompt:

```
Crie snapshot tests para os componentes UI deste projeto usando Vitest + Testing Library. Para cada componente: renderize com props padrao, renderize com cada variacao/variante (size, color, state), renderize com estados especiais (loading, error, empty, disabled) e crie inline snapshots para partes criticas. Configure auto-update de snapshots no script de teste. Use toMatchSnapshot() para o DOM completo e toMatchInlineSnapshot() para partes especificas.
```

Output esperado:

- Snapshots para cada componente
- Variacoes testadas
- Estados especiais cobertos
- Inline snapshots para partes criticas
- Script de update configurado

Dica pro: Snapshot tests detectam mudançãs, não validam corretude — combine com testes de comportamento para cobertura real.

Skill #043 — Gerar Fixtures e Factories de Teste

Categoria: Testing | **Nível:** Intermediario

Quando usar: Quando dados de teste estao repetidos e difíceis de manter entre testes.

Prompt:

```
Crie um sistema de factories e fixtures para os dados de teste deste projeto. Para cada entidade: crie uma factory com faker.js que gera dados aleatorios mas validos, suporte overrides parciais (createUser({ name: 'Custom' })), relacoes entre entidades (createOrder gera user + items automaticamente), traits para variacoes (createUser.admin(), createUser.inactive()), batch creation (createUsers(10)) e funcao de cleanup para testes de integracao.
```

Output esperado:

- Factory por entidade com faker.js
- Overrides parciais
- Relações automáticas
- Traits para variacoes
- Batch creation
- Cleanup functions
- Tipagem completa

Dica pro: Factories com dados aleatorios encontram bugs que dados fixos não encontram — use faker.seed() para reproduzibilidade.

Skill #044 — Criar Mocks para Serviços Externos

Categoria: Testing | **Nível:** Intermediario

Quando usar: Quando testes dependem de APIs externas, email, pagamento ou outros serviços.

Prompt:

```
Crie mocks e stubs para todos os servicos externos deste projeto. Para cada servico: crie uma implementacao fake que simula o comportamento real, suporte respostas configuráveis (sucesso, erro, timeout, rate limit), use MSW (Mock Service Worker) para interceptar HTTP em testes, crie escenarios pre-configurados (paymentSuccess, paymentDeclined, userNotFound) e garanta que a interface e identica ao servico real. Nao use jest.mock() em modulos internos — prefira dependency injection.
```

Output esperado:

- Mocks para cada serviço externo
- MSW handlers configurados
- Cenários pre-configurados
- Respostas de erro simuladas
- Interface idêntica ao real
- Dependency injection configurado

Dica pro: MSW intercepta no nível de network — seus testes exercitam o código real de HTTP, não apenas a lógica de negócio.

Skill #045 — Escrever Property-Based Tests

Categoria: Testing | **Nível:** Avançado

Quando usar: Quando funções tem muitos inputs possíveis e testes manuais não cobrem todos os cenários.

Prompt:

```
Identifique funcoes neste projeto que se beneficiam de property-based testing e crie testes com fast-check. Para cada funcao: defina as propriedades que devem ser verdadeiras para QUALQUER input valido (ex: "parse depois de serialize retorna o original", "sort retorna array do mesmo tamanho", "encrypt seguido de decrypt retorna o original"), gere inputs aleatorios com arbitraries e configure shrinking para encontrar o menor input que falha.
```

Output esperado:

- Funções candidatas identificadas
- Propriedades definidas
- Arbitraries configurados
- Shrinking automático
- Bugs encontrados por inputs inesperados

Dica pro: Comece por funções de parse/serialize, encode/decode, sort/filter — propriedades de round-trip são fáceis de definir.

Skill #046 — Gerar Testes de Contrato de API

Categoria: Testing | **Nível:** Avançado

Quando usar: Quando frontend e backend precisam garantir compatibilidade de API.

Prompt:

Implemente contract testing entre frontend e backend deste projeto. Use Pact ou schema-based testing para: definir contratos de cada endpoint (request shape + response shape), gerar testes no consumer (frontend) que verificam expectativas, gerar testes no provider (backend) que verificam aderência aos contratos, detectar breaking changes automaticamente e integrar no CI para bloquear merges que quebram contratos.

Output esperado:

- Contratos definidos por endpoint
- Testes consumer (frontend)
- Testes provider (backend)
- Detecção de breaking changes
- Integração com CI
- Relatório de compatibilidade

Dica pro: Contract tests substituem testes E2E caros — verifique a interface, não o fluxo completo.

Skill #047 — Criar Relatórios de Cobertura e Preencher Gaps

Categoria: Testing | **Nível:** Básico

Quando usar: Quando precisar identificar e preencher lacunas de cobertura de testes.

Prompt:

Gere um relatório de cobertura de testes deste projeto (vitest --coverage ou jest --coverage). Analise o relatório e identifique: arquivos com 0% de cobertura que são críticos, branches não testados (if/else, switch), funções exportadas sem testes, linhas de error handling não testadas. Priorize por criticidade (auth, payment, data) e gere testes para os top 10 gaps. Meta: cobertura global de 80%+.

Output esperado:

- Relatório de cobertura gerado
- Gaps priorizados por criticidade
- Testes para os 10 gaps mais críticos
- Branches não testados cobertos
- Cobertura subindo para 80%+

Dica pro: 80% de cobertura é o sweet spot — acima disso o ROI cai drasticamente e testes ficam frágeis.

Skill #048 — Escrever Testes de Regressão a Partir de Bug Reports

Categoria: Testing | **Nível:** Básico

Quando usar: Quando um bug é reportado e você quer garantir que nunca mais volte.

Prompt:

Para este bug reportado: [descreva o bug ou cole o ticket]. Crie um teste de regressao que: reproduza o bug exato (o teste deve FALHAR antes do fix), aplique a correcao minima necessaria, verifique que o teste PASSA apos o fix, adicione edge cases relacionados e documente no teste por que ele existe (link para o ticket). Siga o padrao: dado [contexto], quando [acao], entao [resultado esperado].

Output esperado:

- Teste que reproduz o bug (falha antes do fix)
- Fix mínimo aplicado
- Teste passando após fix
- Edge cases adicionais
- Documentação com link do ticket

Dica pro: Sempre escreva o teste ANTES do fix — confirmar que ele falha garante que esta testando o bug certo.

Skill #049 — Gerar Scripts de Teste de Carga

Categoria: Testing | **Nível:** Avancado

Quando usar: Quando precisar verificar como a aplicação se comporta sob carga.

Prompt:

Crie testes de carga para este projeto usando k6 (ou Artillery). Implemente cenarios: smoke test (1 usuario, validar que funciona), load test (100 usuarios, 5 minutos, validar SL0s), stress test (rampup ate 500 usuarios, encontrar breaking point), spike test (0 a 300 usuarios instantaneo). Para cada cenario defina thresholds: p95 latencia < 500ms, error rate < 1%, throughput mínimo. Gere relatorio HTML com graficos.

Output esperado:

- Scripts k6 para 4 cenários
- Thresholds definidos por cenário
- Relatório HTML com gráficos
- Identificacao do breaking point
- Recomendações de otimização

Dica pro: Rode load tests com dados realísticos, não requests identicos — cache mascara problemas de performance.

Skill #050 — Criar Suite de Testes de Acessibilidade

Categoria: Testing | **Nível:** Intermediario

Quando usar: Quando precisar garantir que a interface e acessível a todos os usuários.

Prompt:

Crie uma suite de testes de acessibilidade para este projeto. Inclua: testes automaticos com axe-core integrado ao Vitest/Jest (toHaveNoViolations), testes de navegacao por teclado (Tab, Enter, Escape, Arrow keys), testes de leitor de tela (aria-labels, roles, live regions), testes de contraste de cores, testes de zoom (200%, 400%), verificacao de semantica HTML e checklist WCAG 2.1 AA. Integre no CI para bloquear PRs com violacoes.

Output esperado:

- Testes axe-core por componente
- Testes de navegação por teclado
- Verificação de ARIA
- Teste de contraste
- Teste de zoom
- CI bloqueando violacoes
- Checklist WCAG 2.1 AA

Dica pro: axe-core pega apenas 30% dos problemas — complemente com testes manuais de leitor de tela para as areas criticas.

Capítulo 5 — Git & Version Control

10 skills para dominar Git, PRs, changelogs e workflows de versionamento.

Skill #051 — Escrever Commits Convencionais a Partir de Diffs

Categoria: Git | **Nível:** Básico

Quando usar: Quando precisar de mensagens de commit claras e padronizadas.

Prompt:

Analise as mudancas staged (`git diff --staged`) e escreva uma mensagem de commit seguindo Conventional Commits. Use: `feat:` para features, `fix:` para bugs, `refactor:` para refatoracoes, `docs:` para documentacao, `test:` para testes, `chore:` para manutencao. Inclua scope quando relevante (`feat(auth): ...`), body com contexto do "por que" e footer com breaking changes ou ticket references. Se houver multiplas mudancas distintas, sugira commits separados.

Output esperado:

- Mensagem de commit padronizada
- Tipo correto (`feat/fix/refactor/etc`)
- Scope quando relevante
- Body com contexto
- Sugestão de split se necessário

Dica pro: Um commit deve conter uma única mudança lógica — se precisa de "e" na mensagem, provavelmente são dois commits.

Skill #052 — Gerar Descrições de PR Significativas

Categoria: Git | **Nível:** Básico

Quando usar: Quando precisar criar uma Pull Request bem documentada.

Prompt:

Analise todos os commits desta branch (`git log main..HEAD`) e gere uma descricao de PR completa. Inclua: titulo conciso (< 70 chars), resumo das mudancas em bullets, motivacao/contexto (por que esta mudanca e necessaria), como testar (passo a passo), screenshots se houver mudancas visuais, breaking changes se aplicavel, checklist de review e issues relacionadas (Closes #NNN).

Output esperado:

- Título conciso
- Resumo em bullets

- Motivacao e contexto
- Instruções de teste
- Breaking changes
- Checklist de review
- Issues vinculadas

Dica pro: O resumo deve responder "o que muda para o usuário?" não "quais arquivos mudaram".

Skill #053 — Criar .gitignore Otimizado para sua Stack

Categoria: Git | **Nível:** Básico

Quando usar: Quando iniciar um projeto ou perceber que arquivos indesejados estão sendo commitados.

Prompt:

```
Gere um .gitignore otimizado para este projeto baseado na stack detectada. Inclua: dependencias (node_modules, venv, vendor), build artifacts (dist, .next, __pycache__), environment files (.env, .env.local – mas NAO .env.example), IDE configs (.idea, .vscode/settings.json – mas mantenha .vscode/extensions.json), OS files (.DS_Store, Thumbs.db), logs, cache e arquivos de teste (coverage, playwright-report). Agrupe por categoria com comentarios.
```

Output esperado:

- .gitignore completo por categoria
- Dependências e build artifacts
- Environment files (preservando .example)
- IDE configs (preservando os úteis)
- OS files, logs e cache
- Comentários explicativos

Dica pro: Após criar o .gitignore, rode `git rm -r --cached .` && `git add .` para limpar arquivos já trackeados.

Skill #054 — Resolver Merge Conflicts com Contexto

Categoria: Git | **Nível:** Intermediario

Quando usar: Quando um merge ou rebase gera conflitos que precisam ser resolvidos.

Prompt:

```
Analise os merge conflicts neste projeto e resolva cada um de forma inteligente. Para cada conflito: entenda a intencao de ambas as branches (leia os commits relacionados), escolha a resolucao que preserva a funcionalidade de ambas as mudancas, se ambas as mudancas sao necessarias combine-as corretamente, garanta que o resultado compila e os testes passam. Explique brevemente a decisao tomada em cada conflito.
```

Output esperado:

- Conflitos resolvidos

- Intencao de ambas branches preservada
- Explicacao de cada decisao
- Código compilando
- Testes passando

Dica pro: Sempre rode os testes após resolver conflitos — conflitos semanticos (lógica quebrada sem markers) não aparecem nos marcadores de conflito.

Skill #055 — Gerar Changelogs a Partir do Histórico de Commits

Categoria: Git | **Nível:** Básico

Quando usar: Quando precisar documentar mudanças entre versões para usuários ou stakeholders.

Prompt:

```
Analise o historico de commits desde a ultima tag/release e gere um CHANGELOG.md seguindo o padrao Keep a Changelog.
Agrupe por: Added (features), Changed (alteracoes), Deprecated, Removed, Fixed (bugs), Security. Use os Conventional
Commits para categorizar automaticamente. Inclua links para PRs/issues quando referenciados nos commits. Formate a data
como ISO 8601 e inclua a nova versao seguindo SemVer.
```

Output esperado:

- CHANGELOG.md formatado
- Categorias Keep a Changelog
- Links para PRs/issues
- Versão SemVer calculada
- Data ISO 8601

Dica pro: Gere o changelog automaticamente no CI antes de cada release — nunca escreva manualmente.

Skill #056 — Configurar Git Hooks (Pre-commit, Pre-push)

Categoria: Git | **Nível:** Intermediario

Quando usar: Quando quiser automatizar verificações antes de cada commit ou push.

Prompt:

```
Configure Git hooks completos com Husky + lint-staged para este projeto. Pre-commit: rode ESLint --fix e Prettier nos
arquivos staged, verifique tipos com tsc --noEmit, bloqueie commits de arquivos .env ou com secrets. Pre-push: rode
testes unitarios, verifique se a branch esta atualizada com main. Commit-msg: valide formato Conventional Commits com
commitlint. Instale automaticamente no npm install (prepare script).
```

Output esperado:

- Husky configurado
- Pre-commit com lint + format + type-check

- Pre-push com testes
- Commitlint configurado
- lint-staged otimizado
- Auto-install no npm install

Dica pro: Use lint-staged para rodar lint apenas nos arquivos modificados — rodar no projeto todo é lento demais.

Skill #057 — Criar Documentação de Branching Strategy

Categoria: Git | **Nível:** Básico

Quando usar: Quando o time precisa de um padrão claro para branches e workflow Git.

Prompt:

```
Crie documentacao completa da branching strategy para este projeto. Defina: branch principal (main – always deployable), branches de feature (feature/ticket-123-descricao), branches de bugfix (fix/descricao), branch de release (release/v1.2.0), hotfix flow (hotfix/ → main + backport), naming conventions, merge strategy (squash, rebase ou merge commit e quando usar cada um), branch protection rules recomendadas e diagrama visual do fluxo.
```

Output esperado:

- Documentação da branching strategy
- Naming conventions
- Merge strategy por tipo de branch
- Hotfix flow
- Branch protection rules
- Diagrama visual
- Exemplos praticos

Dica pro: Para times pequenos (< 5 devs), trunk-based development é mais produtivo que Git Flow.

Skill #058 — Analisar Git Blame para Code Ownership

Categoria: Git | **Nível:** Intermediario

Quando usar: Quando precisar entender quem é responsável por cada parte do código.

Prompt:

```
Analise o repositorio e gere um mapa de code ownership. Para cada diretorio/modulo principal, identifique: quem mais contribuiu (por linhas e por commits), quando foi a ultima modificacao significativa, areas orfas (ninguem mexeu ha 6+ meses), areas com muitos contribuidores (risco de conflito). Gere um arquivo CODEOWNERS compativel com GitHub que atribui reviewers automaticamente e um relatorio visual de ownership.
```

Output esperado:

- Mapa de ownership por módulo

- Top contribuidores por area
- Areas órfãs identificadas
- CODEOWNERS file
- Relatório visual

Dica pro: Areas órfãs sem owner são dívida técnica silenciosa — atribua owners mesmo que retroativamente.

Skill #059 — Gerar Release Notes a Partir de PRs

Categoria: Git | **Nível:** Básico

Quando usar: Quando precisar comunicar mudanças de uma nova versão para usuários.

Prompt:

```
Analise todos os PRs merged desde a ultima release e gere release notes voltadas para o USUARIO (nao para devs). Agrupe por: Novidades (features que o usuario percebe), Melhorias (UX, performance), Correcoes (bugs resolvidos), Notas importantes (breaking changes, migracoes necessarias). Use linguagem simples e direta, sem jargao tecnico. Inclua screenshots ou GIFs se houver mudancas visuais referenciadas nos PRs.
```

Output esperado:

- Release notes em linguagem de usuário
- Agrupadas por impacto
- Sem jargao técnico
- Breaking changes destacados
- Links para PRs relevantes

Dica pro: Escreva release notes no presente: "Adicionamos X" não "Foi adicionado X" — soa mais pessoal.

Skill #060 — Configurar Git Worktrees para Desenvolvimento Paralelo

Categoria: Git | **Nível:** Avancado

Quando usar: Quando precisar trabalhar em múltiplas branches simultaneamente sem ficar fazendo stash.

Prompt:

```
Configure Git worktrees para desenvolvimento paralelo neste projeto. Crie um script que: adiciona um novo worktree para qualquer branch (git worktree add), compartilha dependencias instaladas entre worktrees (symlinks para node_modules), sincroniza configuracoes de ambiente, lista worktrees ativos com status e remove worktrees concluidos (cleanup). Inclua aliases git para os comandos mais comuns e documentacao de uso.
```

Output esperado:

- Script de gerenciamento de worktrees

- Compartilhamento de node_modules
- Sync de .env
- Comando de listagem e cleanup
- Git aliases configurados
- Documentação de uso

Dica pro: Worktrees são ideais para code review — abra o PR em um worktree enquanto continua desenvolvendo no principal.

Capítulo 6 — Documentation

12 skills para gerar documentação profissional automaticamente.

Skill #061 — Gerar JSDoc/TSDoc a Partir de Assinaturas de Função

Categoria: Documentation | **Nível:** Básico

Quando usar: Quando funções exportadas não tem documentação inline.

Prompt:

```
Analise todas as funcoes e metodos exportados deste projeto e adicione documentacao JSDoc/TSDoc completa. Para cada funcao: descreva o proposito em uma linha, documente cada parametro (@param) com tipo e descricao, documente o retorno (@returns), adicione @throws para excecoes possiveis, @example com uso real e @see para funcoes relacionadas. Nao documente funcoes privadas ou com nomes auto-explicativos (getId nao precisa de "Gets by ID").
```

Output esperado:

- JSDoc/TSDoc em funções exportadas
- Parâmetros documentados
- Retorno e exceções documentados
- Exemplos de uso
- Funções triviais ignoradas

Dica pro: Bons nomes de função reduzem a necessidade de documentação — renomeie funções confusas em vez de documentar nomes ruins.

Skill #062 — Criar README.md Completo

Categoria: Documentation | **Nível:** Básico

Quando usar: Quando o projeto precisa de um README profissional.

Prompt:

```
Crie um README.md profissional para este projeto. Inclua: nome e descricao curta, badges (build status, coverage, version, license), screenshot ou demo GIF se aplicavel, features principais em bullets, pre-requisitos e instalacao passo a passo, como rodar (dev, testes, build), variaveis de ambiente (tabela), estrutura de pastas simplificada, como contribuir, changelog link e licenca. Use markdown bem formatado com tabela de conteudo.
```

Output esperado:

- README completo e profissional
- Badges configurados

- Instalação passo a passo
- Variáveis de ambiente documentadas
- Estrutura de pastas
- Contributing guide
- Tabela de conteúdo

Dica pro: O README é o primeiro contato de qualquer dev com seu projeto — invista 30 minutos nele.

Skill #063 — Gerar Documentação de API a Partir de Rotas

Categoria: Documentation | **Nível:** Intermediario

Quando usar: Quando sua API precisa de documentação atualizada para consumidores.

Prompt:

```
Analise todas as rotas/endpoints deste projeto e gere documentacao de API completa. Para cada endpoint: metodo HTTP e URL, descricao, parametros de path/query/body com tipos e validacoes, headers obrigatorios (auth), response body com exemplo JSON, status codes possiveis (200, 400, 401, 404, 500), rate limits se aplicavel. Gere em formato Markdown e tambem como OpenAPI 3.1 spec. Inclua exemplos de curl para cada endpoint.
```

Output esperado:

- Documentação por endpoint
- Request/response com exemplos
- Status codes documentados
- OpenAPI 3.1 spec
- Exemplos curl
- Rate limits documentados

Dica pro: Gere a documentação a partir do código, não manualmente — documentação manual fica desatualizada no dia seguinte.

Skill #064 — Criar Architecture Decision Records (ADRs)

Categoria: Documentation | **Nível:** Intermediario

Quando usar: Quando decisoes técnicas importantes precisam ser documentadas para o time futuro.

Prompt:

```
Analise a arquitetura deste projeto e crie ADRs para as decisoes tecnicas mais importantes. Para cada ADR use o formato: Titulo, Status (accepted/deprecated/superseded), Contexto (qual problema estávamos resolvendo), Decisao (o que decidimos), Alternativas consideradas (o que descartamos e por que), Consequencias (trade-offs aceitos). Crie ADRs para: escolha de framework, estrategia de banco, arquitetura de auth, padrao de deploy e estrutura de testes.
```

Output esperado:

- ADRs para decisoes criticas

- Formato padronizado
- Alternativas documentadas
- Trade-offs explicitos
- Numeracao sequencial (001, 002...)

Dica pro: Escreva ADRs NO MOMENTO da decisao — reconstruir o contexto depois e quase impossivel.

Skill #065 — Escrever Guias de Onboarding para Novos Devs

Categoria: Documentation | **Nível:** Básico

Quando usar: Quando novos desenvolvedores entram no time e precisam de contexto.

Prompt:

Crie um guia de onboarding completo para novos desenvolvedores neste projeto. Inclua: como clonar e rodar o projeto (passo a passo com comandos), arquitetura geral (diagrama simplificado), estrutura de pastas explicada, fluxo de uma request (do frontend ao banco), convencoes de codigo, como criar uma feature (branch → code → test → PR), ferramentas necessarias (IDE, extensoes, CLI tools), quem perguntar sobre o que (areas de ownership) e links uteis.

Output esperado:

- Guia passo a passo de setup
- Visao geral da arquitetura
- Estrutura de pastas explicada
- Fluxo de request documentado
- Convencoes do time
- Workflow de feature
- Links e contatos úteis

Dica pro: Peça para o último dev que entrou no time validar o guia — ele sabe exatamente o que faltou.

Skill #066 — Gerar Storybook Stories para Componentes

Categoria: Documentation | **Nível:** Intermediario

Quando usar: Quando precisar documentar e visualizar componentes UI de forma interativa.

Prompt:

Crie Storybook stories para todos os componentes UI reutilizaveis deste projeto. Para cada componente: story Default com props padrao, story para cada variante (size, color, variant), stories para estados especiais (loading, error, disabled, empty), controls interativos para todas as props, actions para callbacks (onClick, onChange), documentacao de uso em MDX e agrupamento por categoria (atoms, molecules, organisms).

Output esperado:

- Stories para cada componente

- Variantes e estados cobertos
- Controls interativos
- Actions configurados
- Documentação MDX
- Organização atomica

Dica pro: Configure Storybook no CI para gerar uma versão estática — serve como documentação viva acessível pelo time.

Skill #067 — Criar Runbooks para Operações Comuns

Categoria: Documentation | **Nível:** Intermediario

Quando usar: Quando operações recorrentes precisam de documentação passo a passo.

Prompt:

```
Crie runbooks para as operacoes mais comuns deste projeto. Inclua runbooks para: deploy manual para producao, rollback de emergencia, investigacao de incidente (coleta de logs, metricas), escalar infraestrutura manualmente, backup e restore do banco, rotacao de secrets/credenciais, onboarding de novo servico/microservice e migracao de banco de dados. Cada runbook deve ter: pre-requisitos, passos numerados com comandos exatos, verificacao de sucesso e troubleshooting.
```

Output esperado:

- Runbook por operação
- Passos numerados com comandos
- Pre-requisitos listados
- Verificação de sucesso em cada passo
- Troubleshooting para erros comuns

Dica pro: Teste cada runbook executando os passos — runbooks não testados falham quando você mais precisa deles.

Skill #068 — Gerar OpenAPI/Swagger Spec a Partir do Código

Categoria: Documentation | **Nível:** Intermediario

Quando usar: Quando precisar de uma spec OpenAPI para geração de clients ou documentação interativa.

Prompt:

```
Analise as rotas deste projeto e gere uma especificacao OpenAPI 3.1 completa. Inclua: info (titulo, versao, descricao), servers (dev, staging, prod), paths com todos os endpoints, schemas para request/response bodies (extraídos dos tipos TypeScript/Zod), security schemes (JWT Bearer), tags para agrupamento e examples para cada endpoint. Configure Swagger UI para servir a documentacao interativa no endpoint /docs.
```

Output esperado:

- Spec OpenAPI 3.1 completa
- Schemas extraídos do código
- Security schemes configurados
- Tags e agrupamento
- Examples por endpoint
- Swagger UI no /docs

Dica pro: Use zod-to-openapi ou typescript-to-openapi para gerar schemas automaticamente dos tipos existentes.

Skill #069 — Escrever Comentários Inline para Lógica Complexa

Categoria: Documentation | **Nível:** Básico

Quando usar: Quando trechos de código tem lógica não-obvia que precisa de contexto.

Prompt:

```
Analise este projeto e identifique trechos de código com lógica complexa, não-obvia ou contra-intuitiva que precisa de comentários. Para cada trecho: adicione um comentário explicando o POR QUE (não o O QUE), referencie issues/tickets se a lógica foi um workaround, use TODO/FIXME/HACK para marcar dívida técnica conhecida. NÃO comente código obvio (// incrementa o contador). Foque em: regras de negócio, workarounds, otimizações e edge cases.
```

Output esperado:

- Comentários em lógica complexa
- Explicação do "por que"
- Referências a tickets
- TODO/FIXME marcados
- Código obvio não comentado

Dica pro: Se você precisa comentar O QUE o código faz, renomeie variáveis e funções primeiro — muitas vezes o comentário se torna desnecessário.

Skill #070 — Criar Documentação de Schema de Banco

Categoria: Documentation | **Nível:** Intermediário

Quando usar: Quando o banco de dados precisa de documentação para devs e DBAs.

Prompt:

```
Analise o schema do banco de dados deste projeto e gere documentação completa. Inclua: diagrama ER (em Mermaid para renderizar no Markdown), tabela por tabela com descrição de cada coluna, relações entre tabelas (1:1, 1:N, N:N), índices e sua justificativa, constraints e regras de negócio no nível do banco, dados de exemplo e queries comuns documentadas. Se usar Prisma, extraia do schema.prisma.
```

Output esperado:

- Diagrama ER em Mermaid
- Documentação por tabela e coluna
- Relações documentadas
- Indexes justificados
- Queries comuns com explicação
- Dados de exemplo

Dica pro: Mantenha o diagrama ER no README — e a visualização mais útil para entender o domínio rapidamente.

Skill #071 — Gerar Guias de Setup de Ambiente

Categoria: Documentation | **Nível:** Básico

Quando usar: Quando precisar documentar como configurar o ambiente de desenvolvimento completo.

Prompt:

```
Crie um guia de setup de ambiente de desenvolvimento completo para este projeto. Inclua: ferramentas necessarias (Node.js, Docker, etc) com versoes e links de instalacao, clone e instalacao de dependencias, configuracao de variaveis de ambiente (com .env.example), setup de banco de dados local (migration + seed), setup de servicos auxiliares (Redis, Elasticsearch via Docker), extensoes de IDE recomendadas, verificacao de setup (script que checa tudo) e troubleshooting de problemas comuns.
```

Output esperado:

- Guia passo a passo completo
- Ferramentas com versões e links
- Setup de banco e serviços
- Script de verificação de setup
- Troubleshooting documentado
- Funciona em Mac, Linux e Windows

Dica pro: Crie um script setup.sh que automatiza tudo — novos devs rodam um comando e estão prontos.

Skill #072 — Escrever Playbooks de Troubleshooting

Categoria: Documentation | **Nível:** Intermediário

Quando usar: Quando erros recorrentes precisam de documentação de diagnóstico.

Prompt:

Crie playbooks de troubleshooting para os problemas mais comuns deste projeto. Para cada problema: sintomas (o que o usuário/dev vê), possíveis causas (ordenadas por probabilidade), passos de diagnóstico (logs para verificar, métricas para checar, queries para rodar), resolução para cada causa, prevenção (como evitar que aconteça de novo). Cubra: app não inicia, conexão com banco falha, deploys falhando, performance degradada, erros de autenticação e integração com terceiros.

Output esperado:

- Playbook por problema comum
- Sintomas claros
- Causas ordenadas por probabilidade
- Passos de diagnóstico
- Resolução por causa
- Prevenção documentada

Dica pro: Atualize os playbooks toda vez que resolver um incidente — conhecimento de post-mortem e ouro.

Capítulo 7 — API Development

12 skills para projetar, construir e manter APIs robustas e escaláveis.

Skill #073 — Design de Estrutura RESTful

Categoria: API Dev | **Nível:** Intermediário

Quando usar: Quando precisar projetar a estrutura de endpoints de uma API do zero.

Prompt:

Projete a estrutura RESTful completa para [domínio do projeto]. Para cada recurso: defina endpoints com metodos HTTP corretos (GET, POST, PUT, PATCH, DELETE), URLs hierarquicas (/users/:id/orders), status codes por operacao, paginacao padrao, filtros e ordenacao via query params, formato de response padronizado (data, meta, errors) e versionamento (URL ou header). Gere a OpenAPI spec e implemente os controllers e routes.

Output esperado:

- Endpoints definidos por recurso
- URLs hierarquicas e consistentes
- Status codes corretos
- Paginação, filtros e ordenação
- Response format padronizado
- OpenAPI spec gerada
- Controllers implementados

Dica pro: Use substantivos no plural para recursos (/users, /orders) e nunca verbos nas URLs (/getUsers e errado).

Skill #074 — Gerar Schemas de Validação Request/Response

Categoria: API Dev | **Nível:** Básico

Quando usar: Quando precisar validar input e tipar output de cada endpoint.

Prompt:

Crie schemas de validacao com Zod para todos os endpoints deste projeto. Para cada endpoint: schema de request body com campos obrigatorios/opcionais e transformacoes, schema de query params com defaults e coercao de tipos, schema de path params com validacao, schema de response para documentacao e type inference. Crie um middleware de validacao generico que aplica os schemas automaticamente e retorna erros 400 com detalhes por campo.

Output esperado:

- Schemas Zod por endpoint
- Request body, query e path params
- Response schemas
- Middleware de validação genérico
- Erros detalhados por campo
- Types inferidos automaticamente

Dica pro: Use `z.coerce` para query params — transforma "123" (string da URL) em 123 (number) automaticamente.

Skill #075 — Criar Middleware Chains (Auth, Logging, Rate-Limit)

Categoria: API Dev | **Nível:** Intermediario

Quando usar: Quando precisar de middlewares reutilizáveis para cross-cutting concerns.

Prompt:

```
Crie uma suite de middlewares para este projeto. Inclua: auth middleware (valida JWT, extrai user, popula req.user), logging middleware (request ID, metodo, URL, status, duracao em ms), rate limiting (por IP e por user, janela deslizando, headers X-RateLimit-*), error handler global (catch-all, formato padronizado, sem stack trace em prod), CORS middleware configuravel por ambiente e request validation middleware (integrado com Zod schemas). Documente a ordem correta de aplicacao.
```

Output esperado:

- Auth middleware com JWT
- Logging com request ID e duracao
- Rate limiting por IP e user
- Error handler global
- CORS por ambiente
- Validation middleware
- Ordem de aplicação documentada

Dica pro: A ordem importa: CORS primeiro, logging, rate-limit, auth, validation, handler. Error handler por último.

Skill #076 — Gerar Estratégia de Versionamento de API

Categoria: API Dev | **Nível:** Avancado

Quando usar: Quando sua API tem consumidores e você precisa evoluir sem quebrar.

Prompt:

Implemente versionamento de API para este projeto. Escolha a estratégia mais adequada: URL path (/v1/users vs /v2/users), header (Accept-Version ou custom), ou content negotiation. Implemente: routing que direciona para a versão correta, versão default para requests sem versão, deprecation headers (Sunset, Deprecation), documentação por versão, migração guide entre versões e testes que verificam backward compatibility.

Output esperado:

- Versionamento implementado
- Routing por versão
- Deprecation headers
- Documentação por versão
- Migration guide
- Testes de backward compatibility

Dica pro: URL path (/v1/) é o mais simples e claro — use a menos que tenha uma razão forte para header-based.

Skill #077 — Criar Webhook Handlers com Retry

Categoria: API Dev | **Nível:** Intermediário

Quando usar: Quando precisar receber ou enviar webhooks de forma confiável.

Prompt:

Implemente um sistema de webhooks completo para este projeto. Recebimento: endpoint de webhook com validação de signature (HMAC-SHA256), idempotência (deduplica por ID do evento), processamento assíncrono (queue), logging de todos os eventos recebidos. Envio: sistema de dispatch com retry exponential backoff (1s, 2s, 4s, 8s, max 5 tentativas), dead letter queue para falhas persistentes, signature nos headers, timeout configurável e dashboard de status por subscription.

Output esperado:

- Endpoint de recebimento com validação
- Idempotência implementada
- Queue para processamento async
- Sistema de envio com retry
- Dead letter queue
- Signature validation
- Dashboard de status

Dica pro: Sempre processe webhooks em background (queue) — responda 200 imediatamente e processe depois.

Skill #078 — Gerar Implementações de Paginação

Categoria: API Dev | **Nível:** Intermediario

Quando usar: Quando endpoints retornam listas que podem ser grandes.

Prompt:

```
Implemente paginacao em todos os endpoints de listagem deste projeto. Suporte dois modos: offset-based (?page=2&limit=20) para UIs simples e cursor-based (?cursor=abc123&limit=20) para feeds/scroll infinito. Inclua: metadata na response (total, page, totalPages, hasMore, nextCursor), headers de Link (RFC 5988) para navegacao, limite maximo configuravel (prevenir limit=999999), ordenacao (sort=name:asc,createdAt:desc) e helper/wrapper reutilizavel que qualquer endpoint pode usar.
```

Output esperado:

- Paginação offset-based e cursor-based
- Metadata completa na response
- Headers de Link RFC 5988
- Limite máximo configurável
- Ordenação flexível
- Helper reutilizável

Dica pro: Use cursor-based para dados que mudam frequentemente (feeds) — offset-based pula items quando dados são inseridos entre páginas.

Skill #079 — Criar Endpoints de Upload de Arquivos

Categoria: API Dev | **Nível:** Intermediario

Quando usar: Quando precisar de upload de arquivos com validação e armazenamento.

Prompt:

```
Implemente upload de arquivos seguro para este projeto. Inclua: endpoint multipart/form-data com multer ou busboy, validacao de tipo MIME (whitelist, nao confie na extensao), limite de tamanho configuravel, scan de virus/malware (ClamAV) se possivel, armazenamento em S3/GCS com URLs assinadas, geracao de thumbnails para imagens (sharp), progresso de upload para o frontend, cleanup de arquivos orfao e testes de upload com diferentes tipos de arquivo.
```

Output esperado:

- Endpoint de upload multipart
- Validação de MIME type e tamanho
- Armazenamento em cloud (S3/GCS)
- URLs assinadas para acesso
- Thumbnails para imagens
- Cleanup de órfãos
- Testes de upload

Dica pro: NUNCA confie na extensão do arquivo — valide o MIME type pelos magic bytes do arquivo.

Skill #080 — Gerar Error Handling Padronizado

Categoria: API Dev | **Nível:** Básico

Quando usar: Quando erros da API retornam formatos inconsistentes.

Prompt:

```
Implemente error handling padronizado em toda a API deste projeto. Crie: classe AppError com code, message, statusCode e details, error handler global que captura todas as excecoes, formato de resposta padronizado ({ error: { code, message, details } }), mapeamento de erros conhecidos (ValidationError → 400, NotFoundError → 404, UnauthorizedError → 401), logging de erros (sem dados sensiveis), stack trace apenas em dev e codigos de erro documentados para o frontend.
```

Output esperado:

- Classe AppError com hierarquia
- Error handler global
- Response format padronizado
- Mapeamento de erros → status codes
- Logging sem dados sensíveis
- Códigos de erro documentados

Dica pro: Crie códigos de erro específicos (USER_NOT_FOUND, EMAIL_TAKEN) — o frontend precisa deles para exibir mensagens corretas.

Skill #081 — Criar Endpoints Real-Time com SSE/WebSocket

Categoria: API Dev | **Nível:** Avançado

Quando usar: Quando precisar de comunicação em tempo real entre servidor e cliente.

Prompt:

```
Implemente comunicacao real-time neste projeto. Para notificacoes simples: use Server-Sent Events (SSE) com endpoint GET /events, reconnect automatico, event types customizados e last-event-id para recuperar eventos perdidos. Para comunicacao bidirecional: use WebSocket com rooms/channels, autenticao no handshake, heartbeat a cada 30s, reconexao com exponential backoff no client, buffer de mensagens offline e tipagem de mensagens. Escolha SSE ou WS baseado na necessidade.
```

Output esperado:

- SSE ou WebSocket implementado
- Autenticação integrada
- Reconnect automático
- Heartbeat/keep-alive
- Buffer de mensagens offline
- Tipagem de mensagens
- Testes de conexão

Dica pro: Use SSE para notificações server→client (mais simples, funciona com HTTP/2). Use WebSocket apenas se precisar de comunicação bidirecional.

Skill #082 — Gerar Rate Limiting por Endpoint

Categoria: API Dev | **Nível:** Intermediario

Quando usar: Quando precisar proteger endpoints contra abuso e garantir fair usage.

Prompt:

```
Implemente rate limiting granular neste projeto. Configure: limites diferentes por endpoint (login: 5/15min, API geral: 100/min, upload: 10/hora), identificacao por IP e por user autenticado, algoritmo sliding window (Redis) para precisao, headers de resposta (X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset), response 429 com Retry-After, whitelist para IPs internos/health checks e dashboard de rate limit usage.
```

Output esperado:

- Rate limits por endpoint
- Sliding window com Redis
- Headers X-RateLimit-*
- Response 429 com Retry-After
- Whitelist configurável
- Dashboard de uso

Dica pro: Rate limit o login mais agressivamente (5/15min por IP) — e o endpoint mais atacado.

Skill #083 — Criar Endpoints Batch/Bulk

Categoria: API Dev | **Nível:** Intermediario

Quando usar: Quando clientes precisam criar/atualizar/deletar múltiplos recursos de uma vez.

Prompt:

```
Implemente endpoints batch/bulk para este projeto. Crie: POST /resources/batch para criacao em lote, PATCH /resources/batch para atualizacao em lote, DELETE /resources/batch para delecao em lote. Cada endpoint deve: aceitar array de operacoes, processar em transacao (all-or-nothing) ou parcialmente (com relatorio de sucesso/falha por item), limitar tamanho do batch (max 100 items), validar cada item individualmente e retornar resultado detalhado por item.
```

Output esperado:

- Endpoints batch para CRUD
- Processamento em transação
- Limite de batch configurável
- Validação por item
- Resultado detalhado por item
- Modo all-or-nothing e parcial

Dica pro: Ofereça ambos os modos: all-or-nothing (transação) para consistência e parcial para tolerância a falhas.

Skill #084 — Gerar Estratégias de Cache de API

Categoria: API Dev | **Nível:** Avançado

Quando usar: Quando endpoints retornam dados que podem ser cacheados para reduzir carga.

Prompt:

```
Implemente caching estrategico na API deste projeto. Nivel HTTP: Cache-Control headers por endpoint (imutaveis para assets, stale-while-revalidate para dados), ETag com validacao 304 Not Modified. Nivel aplicacao: cache Redis com TTL por tipo de dado, invalidacao automatica em write operations (cache-aside pattern), cache key generation consistente, cache warming para dados frequentes. Inclua metricas de hit/miss ratio e bypass de cache para debug (?nocache=1 em dev).
```

Output esperado:

- Cache-Control headers por endpoint
- ETag com 304 Not Modified
- Redis cache com TTL
- Invalidacao em writes
- Cache warming
- Métricas hit/miss
- Debug bypass

Dica pro: Cache invalidation é o problema mais difícil — prefira TTL curto (60s) com stale-while-revalidate a invalidacao manual.

Capítulo 8 — Database

10 skills para modelar, consultar e otimizar bancos de dados.

Skill #085 — Gerar Queries SQL a Partir de Linguagem Natural

Categoria: Database | **Nível:** Básico

Quando usar: Quando precisar escrever queries complexas descrevendo o que quer em português.

Prompt:

Gere a query SQL para: [descreva o que precisa em linguagem natural, ex: "todos os usuarios que fizeram mais de 3 compras no ultimo mes, agrupados por cidade, com o valor total gasto"]. Use o schema deste projeto como referencia. A query deve: usar JOINS explicitos (nao subqueries correlacionadas quando possivel), incluir apenas as colunas necessarias, ter aliases descritivos, estar otimizada para o volume estimado de dados e incluir EXPLAIN ANALYZE para verificar o plano.

Output esperado:

- Query SQL otimizada
- JOINS explicitos
- Aliases descritivos
- EXPLAIN ANALYZE
- Comentários explicando partes complexas

Dica pro: Sempre informe o volume estimado de dados — uma query ótima para 1K rows pode ser pessima para 1M.

Skill #086 — Criar Indexes a Partir de Análise de Queries

Categoria: Database | **Nível:** Intermediario

Quando usar: Quando queries estao lentas e precisam de indexes para melhorar performance.

Prompt:

Analise todas as queries deste projeto (ORM queries, raw SQL, query builders) e recomende indexes. Para cada index sugerido: mostre a query que se beneficia, o EXPLAIN sem index (seq scan), o EXPLAIN com index (index scan), tipo de index (B-tree, GIN, GiST, hash), se deve ser composto (multi-column) e a ordem das colunas, e o impacto estimado no write performance. Crie as migrations para adicionar os indexes.

Output esperado:

- Queries lentas identificadas
- Indexes recomendados com justificativa

- EXPLAIN antes/depois
- Tipo de index adequado
- Migrations criadas
- Impacto em writes documentado

Dica pro: Indexes compostos devem seguir a regra: colunas de igualdade primeiro, depois range, depois ORDER BY.

Skill #087 — Gerar Scripts de Seed Data

Categoria: Database | **Nível:** Básico

Quando usar: Quando precisar de dados realistas para desenvolvimento e testes.

Prompt:

```
Crie scripts de seed completos para este projeto. Gere dados realistas (nao "test123") usando faker.js: usuarios com nomes, emails e avatares plausiveis, relacionamentos entre entidades consistentes (orders pertencem a users reais), volume configuravel (dev: 50 records, staging: 1000, load-test: 100k), dados que cobrem edge cases (usuario sem pedidos, pedido cancelado, produto sem estoque) e um usuario admin com credenciais conhecidas para dev. Scripts devem ser idempotentes (rodar multiplas vezes sem duplicar).
```

Output esperado:

- Seeds com dados realistas (faker.js)
- Relacionamentos consistentes
- Volume configurável por ambiente
- Edge cases cobertos
- Usuário admin para dev
- Idempotência garantida

Dica pro: Inclua um seed de "golden path" — dados que permitem testar o fluxo principal completo sem setup manual.

Skill #088 — Criar Estratégias de Rollback de Migration

Categoria: Database | **Nível:** Avançado

Quando usar: Quando precisar reverter mudanças no banco de forma segura.

Prompt:

```
Revise todas as migrations deste projeto e garanta que cada uma tem um rollback seguro. Para cada migration: verifique que a down() reverte completamente a up(), trate casos criticos (drop column com dados, rename table), crie migration de dados quando necessario (backfill antes de NOT NULL), adicione checkpoints em migrations longas, documente dependencias entre migrations e crie um runbook de rollback de emergencia com comandos exatos.
```

Output esperado:

- Downs revisadas e corrigidas
- Data migrations onde necessário
- Checkpoints em migrations longas
- Dependências documentadas
- Runbook de rollback
- Testes de rollback

Dica pro: Sempre teste a `down()` em dev antes de rodar a `up()` em produção — downs não testadas falham quando você mais precisa.

Skill #089 — Gerar Stored Procedures a Partir de Lógica de Negocio

Categoria: Database | **Nível:** Avancado

Quando usar: Quando lógica complexa precisa rodar no banco para performance ou atomicidade.

Prompt:

Identifique operacoes neste projeto que se beneficiam de stored procedures e implemente-as. Candidatas: calculos agregados complexos, operacoes atomicas multi-tabela, transformacoes de dados em batch, triggers para auditoria. Para cada procedure: crie com parametros tipados, tratamento de erros, transacao explicita, logging de execucao, testes com dados de exemplo e uma funcao wrapper na aplicacao que chama a procedure com tipagem TypeScript.

Output esperado:

- Stored procedures criadas
- Parâmetros tipados
- Transações explícitas
- Error handling
- Wrapper tipado na aplicação
- Testes de procedures

Dica pro: Use stored procedures apenas para atomicidade e performance critica — lógica de negocio no banco é mais difícil de testar e versionar.

Skill #090 — Criar Scripts de Backup e Restore

Categoria: Database | **Nível:** Intermediario

Quando usar: Quando precisar de estratégia de backup automatizado para o banco.

Prompt:

Crie um sistema de backup automatizado para o banco de dados deste projeto. Inclua: script de backup comprimido (`pg_dump/mysqldump + gzip`), upload automatico para S3/GCS com retencao (diario 7d, semanal 4w, mensal 12m), script de restore com verificacao de integridade, backup antes de cada migration automaticamente, teste de restore periodico (backup nao testado nao e backup), notificacao de sucesso/falha (Slack/email) e cron job ou GitHub Action para automacao.

Output esperado:

- Script de backup comprimido
- Upload para cloud com retenção
- Script de restore com verificação
- Backup pre-migration
- Teste automático de restore
- Notificação de status
- Automação com cron/Actions

Dica pro: Um backup que você nunca testou o restore NAO é um backup — teste o restore pelo menos mensalmente.

Skill #091 — Gerar Configurações de Connection Pooling

Categoria: Database | **Nível:** Intermediario

Quando usar: Quando conexões com o banco estão sendo criadas/destruídas excessivamente.

Prompt:

```
Configure connection pooling otimizado para o banco de dados deste projeto. Se Node.js: configure o pool do Prisma/Knex/pg com min/max connections, idle timeout, acquire timeout e statement timeout. Se necessário, configure PgBouncer como middleware de pooling. Dimensione baseado em: num_cores * 2 + 1 para max connections, monitore pool usage com métricas, configure retry para conexões falhas e alerte quando pool atingir 80% de capacidade.
```

Output esperado:

- Pool configurado e dimensionado
- Timeouts definidos
- PgBouncer (se necessário)
- Métricas de pool usage
- Retry para falhas
- Alertas de capacidade

Dica pro: Menos conexões no pool é melhor — cada conexão consome ~10MB de memória no PostgreSQL.

Skill #092 — Criar Patterns de Multi-Tenancy

Categoria: Database | **Nível:** Avancado

Quando usar: Quando sua aplicação serve múltiplos clientes/organizações com dados isolados.

Prompt:

Implemente multi-tenancy neste projeto. Escolha a estratégia mais adequada: schema por tenant (isolamento forte), tabela compartilhada com tenant_id (simples), ou banco separado por tenant (isolamento total). Implemente: middleware que identifica o tenant (subdomain, header, token), filtro automático em todas as queries (tenant_id = current_tenant), prevenção de data leak (testes que verificam isolamento), migrations que rodam para todos os tenants e seed por tenant.

Output esperado:

- Estratégia de multi-tenancy implementada
- Middleware de identificação de tenant
- Filtro automático em queries
- Prevenção de data leak
- Migrations multi-tenant
- Testes de isolamento

Dica pro: Comece com tabela compartilhada + tenant_id — e a mais simples e migrar para schema separado depois e possível.

Skill #093 — Gerar Recomendações de Otimização de Queries

Categoria: Database | **Nível:** Intermediario

Quando usar: Quando o banco está lento e você precisa de um diagnóstico completo.

Prompt:

Faca um diagnostico completo de performance do banco de dados deste projeto. Analise: queries mais lentas (slow query log), queries mais frequentes (identifique hot queries), uso de indexes (quais estao sendo usados, quais nao), table bloat e necessidade de VACUUM, configuracoes do banco (shared_buffers, work_mem, effective_cache_size), locks e deadlocks. Para cada problema: recomende a otimizacao com impacto estimado e implemente as top 5.

Output esperado:

- Queries lentas identificadas
- Uso de indexes analisado
- Configurações revisadas
- Top 5 otimizações implementadas
- Impacto estimado por otimização
- Monitoramento configurado

Dica pro: pg_stat_statements é a melhor ferramenta para identificar queries problematicas em PostgreSQL — ative se ainda não estiver.

Skill #094 — Criar Validação de Dados na Camada do Banco

Categoria: Database | **Nível:** Intermediario

Quando usar: Quando precisar de validações no banco como última linha de defesa.

Prompt:

```
Implemente validacoes na camada do banco de dados como ultima linha de defesa. Adicione: CHECK constraints para regras de negocio (preco > 0, status IN ('active', 'inactive')), UNIQUE constraints compostos onde necessario, NOT NULL com defaults sensatos, triggers para validacoes complexas (ex: estoque nao pode ficar negativo), foreign keys com ON DELETE correto (CASCADE, SET NULL, RESTRICT) e partial indexes para constraints condicionais. Documente cada constraint com COMMENT ON.
```

Output esperado:

- CHECK constraints para regras de negocio
- UNIQUE constraints compostos
- NOT NULL com defaults
- Triggers para validações complexas
- Foreign keys com ON DELETE correto
- COMMENT ON para documentação

Dica pro: Validação no banco é a última defesa — valide na aplicação também, mas nunca confie que a aplicação vai validar sempre.

Capítulo 9 — DevOps & Deploy

10 skills para automatizar infraestrutura, CI/CD e deploy com Claude Code.

Skill #095 — Gerar Dockerfile Otimizado para Produção

Categoria: DevOps | **Nível:** Intermediario

Quando usar: Quando precisar containerizar sua aplicação com boas práticas de produção.

Prompt:

Analise este projeto e crie um Dockerfile otimizado para producao. Use multi-stage build, minimize o tamanho da imagem, configure usuario nao-root, adicione health check, use .dockerignore apropriado e cache layers de dependencias. Inclua comentarios explicando cada decisao.

Output esperado:

- Dockerfile com multi-stage build
- .dockerignore configurado
- Imagem final mínima (alpine/distroless)
- Health check configurado
- Usuário não-root
- Cache de dependências otimizado

Dica pro: Mencione o runtime específico (Node, Python, Go) para obter otimizacoes específicas da linguagem.

Skill #096 — Criar Pipeline CI/CD com GitHub Actions

Categoria: DevOps | **Nível:** Intermediario

Quando usar: Quando precisar automatizar build, testes e deploy com GitHub Actions.

Prompt:

Crie uma pipeline CI/CD completa com GitHub Actions para este projeto. Inclua: lint, testes unitarios, testes de integracao, build, analise de seguranca com trivy/snyk, deploy para staging em PRs e deploy para producao em push na main. Use cache de dependencias, matrix strategy para multiplas versoes e secrets do GitHub. Crie os workflows em .github/workflows/.

Output esperado:

- Workflow de CI (lint + testes + build)
- Workflow de CD (deploy staging + produção)

- Cache de dependências configurado
- Matrix strategy para versões
- Secrets e environments configurados

Dica pro: Especifique o provider de deploy (Vercel, AWS, GCP) para obter steps específicos.

Skill #097 — Gerar Manifests Kubernetes

Categoria: DevOps | **Nível:** Avançado

Quando usar: Quando precisar deployar sua aplicação em Kubernetes.

Prompt:

Gere manifests Kubernetes completos para este projeto. Inclua: Deployment com rolling update, Service (ClusterIP + LoadBalancer), Ingress com TLS, ConfigMap para configuracoes, Secret para credenciais, HPA para auto-scaling, PDB para disponibilidade, ResourceQuotas e NetworkPolicy. Organize em kustomize com overlays para dev/staging/prod.

Output esperado:

- Deployment com probes e resources
- Service e Ingress com TLS
- ConfigMap e Secret
- HPA e PDB
- Kustomize com overlays por ambiente
- NetworkPolicy

Dica pro: Informe o tamanho esperado de tráfego para que o Claude dimensione os resources corretamente.

Skill #098 — Criar Infrastructure as Code (Terraform)

Categoria: DevOps | **Nível:** Avançado

Quando usar: Quando precisar provisionar infraestrutura de forma reproduzível.

Prompt:

Crie modulos Terraform para a infraestrutura completa deste projeto na AWS. Inclua: VPC com subnets publicas/privadas, ECS Fargate ou EKS para containers, RDS PostgreSQL com replica de leitura, ElastiCache Redis, ALB com certificado ACM, Route53, S3 para assets, CloudWatch para logs e alarmes. Use modulos reutilizaveis, remote state no S3 e workspaces para ambientes.

Output esperado:

- Módulos Terraform organizados
- VPC, ECS/EKS, RDS, Redis
- ALB, Route53, S3

- Remote state e workspaces
- Variaveis e outputs documentados
- tfvars por ambiente

Dica pro: Substitua AWS pelo seu cloud provider preferido (GCP, Azure) para obter recursos equivalentes.

Skill #099 — Gerar Configurações de Monitoring e Alertas

Categoria: DevOps | **Nível:** Intermediario

Quando usar: Quando precisar configurar observabilidade completa para sua aplicação.

Prompt:

```
Configure monitoring completo para este projeto. Inclua: metricas customizadas com Prometheus (ou Datadog/New Relic), dashboards Grafana com paineis de latencia/throughput/errors/saturation (RED/USE), alertas para SLOs criticos (latencia p99 > 500ms, error rate > 1%, CPU > 80%), integracao com Slack/PagerDuty e runbooks para cada alerta. Crie o docker-compose para subir a stack localmente.
```

Output esperado:

- Instrumentacao de métricas na aplicação
- Dashboards Grafana (JSON)
- Regras de alertas Prometheus
- Docker-compose para stack de monitoring
- Runbooks por alerta
- Integração com notificações

Dica pro: Especifique seus SLOs reais para alertas mais relevantes ao seu contexto.

Skill #100 — Criar Deploy Blue-Green

Categoria: DevOps | **Nível:** Avancado

Quando usar: Quando precisar de deploys com zero downtime e rollback instantaneo.

Prompt:

```
Implemente uma estrategia de deploy blue-green para este projeto. Crie scripts que: provisionem o ambiente "green" identico ao "blue", executem smoke tests automaticos no green, facam o switch de trafego via load balancer, monitorem metricas pos-switch por 5 minutos e facam rollback automatico se error rate subir. Inclua scripts de cleanup do ambiente antigo.
```

Output esperado:

- Scripts de provisioning blue/green
- Smoke tests automatizados
- Script de switch de tráfego

- Monitoramento pos-deploy
- Rollback automático
- Cleanup scripts

Dica pro: Combine com a Skill #096 para integrar blue-green no pipeline CI/CD.

Skill #101 — Gerar Configuração Nginx / Reverse Proxy

Categoria: DevOps | **Nível:** Intermediario

Quando usar: Quando precisar configurar Nginx como reverse proxy, load balancer ou servidor de assets.

Prompt:

```
Crie configuracao Nginx completa para este projeto como reverse proxy. Inclua: upstream com load balancing (round-robin), SSL/TLS com Let's Encrypt (certbot), compressao gzip/brotli, cache de assets estaticos com headers corretos, rate limiting por IP, security headers (HSTS, CSP, X-Frame-Options), redirect HTTP para HTTPS e configuracao de WebSocket se aplicavel. Inclua docker-compose com auto-renewal de certificado.
```

Output esperado:

- nginx.conf completo e otimizado
- Configuração SSL/TLS
- Rate limiting e security headers
- Cache de assets
- Docker-compose com certbot
- Configuração de upstream

Dica pro: Informe se precisa de WebSocket support ou streaming para configurações específicas.

Skill #102 — Criar Setup de Agregação de Logs

Categoria: DevOps | **Nível:** Intermediario

Quando usar: Quando precisar centralizar e analisar logs de múltiplos serviços.

Prompt:

```
Configure uma stack de agregacao de logs para este projeto. Implemente: logging estruturado (JSON) na aplicacao com correlation IDs, stack ELK (Elasticsearch + Logstash + Kibana) ou Loki + Grafana, rotacao e retencao de logs, dashboards para busca e analise, alertas para padroes de erro e filtros por servico/nivel/request-id. Inclua docker-compose para rodar localmente.
```

Output esperado:

- Logging estruturado na aplicação
- Docker-compose com stack de logs
- Dashboards de busca e análise

- Rotação e retenção configurados
- Alertas para padrões de erro
- Correlation ID implementado

Dica pro: Use Loki + Grafana para uma stack mais leve, ELK para buscas full-text mais poderosas.

Skill #103 — Gerar Health Check Endpoints e Probes

Categoria: DevOps | **Nível:** Básico

Quando usar: Quando precisar de endpoints de health check para orquestradores e load balancers.

Prompt:

```
Crie endpoints de health check completos para este projeto. Implemente: /health (liveness - resposta rapida), /ready (readiness - verifica dependencias: DB, Redis, APIs externas), /metrics (metricas Prometheus), /info (versao, commit, uptime). Cada check deve ter timeout configuravel, retornar status detalhado por dependencia e nao expor informacoes sensiveis em producao. Adicione middleware de graceful shutdown.
```

Output esperado:

- Endpoint /health (liveness probe)
- Endpoint /ready (readiness probe)
- Endpoint /metrics (Prometheus)
- Endpoint /info (versão e metadata)
- Graceful shutdown implementado
- Testes para cada endpoint

Dica pro: O /health deve ser ultra-rápido (sem I/O). Coloque verificações de dependências apenas no /ready.

Skill #104 — Criar Configurações de Auto-Scaling

Categoria: DevOps | **Nível:** Avançado

Quando usar: Quando precisar que sua aplicação escale automaticamente baseado em demanda.

Prompt:

```
Configure auto-scaling completo para este projeto. Implemente: HPA no Kubernetes (ou equivalente no seu cloud) com metricas customizadas (CPU, memoria, requests por segundo, latencia), scale-to-zero para ambientes de dev, warm-up period e cooldown configurados, pre-scaling para eventos agendados e alertas quando atingir limites. Inclua testes de carga com k6 para validar o scaling.
```

Output esperado:

- HPA com métricas customizadas
- Scale-to-zero para dev
- Warm-up e cooldown configurados

- Scripts de teste de carga (k6)
- Alertas de limites
- Documentação de thresholds

Dica pro: Configure cooldown periods generosos (5-10 min) para evitar flapping de instâncias.

Capítulo 10 — Security

10 skills para blindar sua aplicação contra vulnerabilidades e ataques.

Skill #105 — Audit de Segurança OWASP

Categoria: Security | **Nível:** Avançado

Quando usar: Quando precisar verificar seu codebase contra as vulnerabilidades mais comuns da web.

Prompt:

```
Faca um audit de segurança completo neste projeto baseado no OWASP Top 10. Para cada categoria (Injection, Broken Auth, Sensitive Data Exposure, XXE, Broken Access Control, Security Misconfig, XSS, Insecure Deserialization, Known Vulnerabilities, Insufficient Logging), analise o código e reporte: vulnerabilidades encontradas com severity (critica/alta/media/baixa), arquivo e linha exatos, descrição do risco e o fix recomendado. Corrija automaticamente todas as vulnerabilidades criticas.
```

Output esperado:

- Relatório completo por categoria OWASP
- Vulnerabilidades com severity e localização
- Fixes automáticos para críticas
- Recomendações para médias e baixas
- Checklist de verificação

Dica pro: Execute esta skill periodicamente (a cada sprint) como parte do ciclo de segurança.

Skill #106 — Gerar Sanitização e Validação de Input

Categoria: Security | **Nível:** Básico

Quando usar: Quando precisar validar e sanitizar todos os inputs do usuário.

Prompt:

```
Analise todos os pontos de entrada de dados do usuário neste projeto (formularios, query params, body de requests, headers, uploads). Para cada ponto, implemente: validacao com Zod/Joi/class-validator, sanitizacao contra XSS (escape HTML), limitacao de tamanho, validacao de tipo e formato, mensagens de erro claras sem expor internals. Crie um middleware centralizado de validacao reutilizavel.
```

Output esperado:

- Schemas de validação para cada endpoint
- Middleware centralizado de validação
- Sanitização contra XSS

- Limites de tamanho configurados
- Mensagens de erro padronizadas
- Testes de validação

Dica pro: Use Zod para projetos TypeScript — ele infere os tipos automaticamente dos schemas.

Skill #107 — Criar Fluxo de Autenticação (JWT + OAuth)

Categoria: Security | **Nível:** Intermediario

Quando usar: Quando precisar implementar autenticação segura na sua aplicação.

Prompt:

```
Implemente um sistema de autenticação completo e seguro para este projeto. Inclua: registro com validação de email, login com JWT (access token 15min + refresh token 7dias), armazenamento seguro de senhas (bcrypt cost 12), OAuth 2.0 com Google e GitHub, middleware de autenticação, blacklist de tokens no logout, rate limiting no login (5 tentativas/15min), proteção contra timing attacks e testes completos.
```

Output esperado:

- Endpoints de register, login, logout, refresh
- OAuth 2.0 (Google + GitHub)
- JWT com access + refresh tokens
- Hash de senhas com bcrypt
- Rate limiting no login
- Middleware de auth
- Testes completos

Dica pro: Nunca armazene JWTs no localStorage — use httpOnly cookies com SameSite=Strict.

Skill #108 — Gerar Content Security Policy Headers

Categoria: Security | **Nível:** Intermediario

Quando usar: Quando precisar proteger sua aplicação contra XSS e injeção de conteúdo.

Prompt:

```
Configure todos os security headers necessarios para este projeto. Implemente: Content-Security-Policy (restritiva, com nonces para inline scripts), Strict-Transport-Security (HSTS com preload), X-Content-Type-Options, X-Frame-Options, Referrer-Policy, Permissions-Policy (desabilitar camera/mic/geolocation se nao necessarios), CORS configurado por ambiente. Crie middleware que aplique todos os headers e testes que verifiquem cada um.
```

Output esperado:

- Middleware de security headers
- CSP restritiva com nonces

- HSTS com preload
- CORS por ambiente
- Permissions-Policy
- Testes para cada header

Dica pro: Comece com CSP em mode report-only para identificar violacoes antes de enforçar.

Skill #109 — Criar Gestao de Secrets

Categoria: Security | **Nível:** Intermediario

Quando usar: Quando precisar gerenciar credenciais e secrets de forma segura.

Prompt:

Implemente um sistema de gestao de secrets para este projeto. Configure: .env por ambiente (dev/staging/prod) com validacao de variaveis obrigatorias na inicializacao, integracao com vault (HashiCorp Vault ou AWS Secrets Manager), rotacao automatica de secrets, .env.example documentado sem valores reais, pre-commit hook que bloqueia commit de arquivos .env e scan do historico git para secrets vazados. Crie um script de setup para novos devs.

Output esperado:

- .env por ambiente com validação
- Integração com secrets manager
- Pre-commit hook anti-secrets
- .env.example documentado
- Script de setup para devs
- Scan de histórico git

Dica pro: Use git-secrets ou truffleHog para scanear o histórico antes de tornar um repo publico.

Skill #110 — Gerar Prevenção de SQL Injection

Categoria: Security | **Nível:** Básico

Quando usar: Quando precisar garantir que queries ao banco estao protegidas contra injection.

Prompt:

Analise todas as queries SQL e interacoes com banco de dados neste projeto. Identifique: queries com concatenacao de strings (vulneraveis), inputs nao sanitizados usados em queries, stored procedures sem parametrizacao. Para cada caso, refatore para usar prepared statements/parameterized queries. Se estiver usando um ORM, verifique se ha raw queries vulneraveis. Gere testes que tentem injetar SQL malicioso para validar a protecao.

Output esperado:

- Lista de queries vulneráveis encontradas
- Refatoracao para prepared statements

- Testes de injeção SQL
- Verificação de raw queries no ORM
- Relatório de cobertura

Dica pro: Mesmo usando ORMs, cuidado com métodos `.raw()` ou `.query()` que aceitam strings — sempre parametrize.

Skill #111 — Criar Estratégias de Prevenção XSS

Categoria: Security | **Nível:** Intermediário

Quando usar: Quando precisar proteger sua aplicação contra Cross-Site Scripting.

Prompt:

```
Implemente protecao completa contra XSS neste projeto. Analise todos os pontos onde dados do usuario sao renderizados e implemente: escape automatico de output (HTML entities), sanitizacao de HTML rico com DOMPurify ou similar, CSP com nonces para scripts inline, validacao de URLs contra javascript: protocol, protecao de cookies com httpOnly/secure/sameSite e sanitizacao em formularios. Crie testes com payloads XSS comuns.
```

Output esperado:

- Escape automático de output
- Sanitização de HTML rico
- CSP implementada
- Cookies protegidos
- Testes com payloads XSS
- Middleware de sanitização

Dica pro: Use frameworks que fazem escape por padrão (React, Vue) e cuidado com `dangerouslySetInnerHTML`.

Skill #112 — Gerar Modelo de Autorização RBAC/ABAC

Categoria: Security | **Nível:** Avançado

Quando usar: Quando precisar controlar permissões granulares de usuários.

Prompt:

```
Implemente um sistema de autorizacao RBAC (Role-Based Access Control) completo para este projeto. Crie: tabelas/models para roles e permissions, middleware de autorizacao que verifica permissoes por rota, decorators/guards para proteger endpoints, roles padrao (admin, editor, viewer), sistema de heranca de permissoes, audit log de mudancas de permissao e painel admin para gerenciar roles. Se necessario, evolua para ABAC com policiees baseadas em atributos.
```

Output esperado:

- Models de roles e permissions
- Middleware de autorização

- Guards/decorators por endpoint
- Roles padrão configurados
- Herança de permissões
- Audit log
- Testes de autorização

Dica pro: Comece com RBAC simples. So migre para ABAC quando as regras de negocio exigirem policiees complexas.

Skill #113 — Criar Mecanismo de Rotação de API Keys

Categoria: Security | **Nível:** Avancado

Quando usar: Quando precisar gerenciar API keys de forma segura com rotação automática.

Prompt:

```
Implemente um sistema de API key management para este projeto. Inclua: geracao de API keys com prefixo identificavel (sk_live_, sk_test_), armazenamento com hash (nunca plain text), suporte a multiplas keys por usuario, rotacao de keys sem downtime (grace period de 24h para key antiga), rate limiting por key, scopes/permissoes por key, revogacao instantanea, dashboard de uso por key e webhook de notificacao pre-expiracao.
```

Output esperado:

- Geração de API keys seguras
- Armazenamento com hash
- Rotação com grace period
- Rate limiting por key
- Scopes e permissões
- Dashboard de uso
- Testes completos

Dica pro: Use o formato prefixo + random bytes (ex: sk_live_a1b2c3) para facilitar identificacao e debug.

Skill #114 — Gerar Checklist de Audit de Segurança

Categoria: Security | **Nível:** Intermediario

Quando usar: Antes de cada release ou deploy para produção.

Prompt:

```
Gere um checklist de seguranca completo e automatizado para este projeto. Para cada item, crie um script ou teste que verifica automaticamente: dependencias com vulnerabilidades conhecidas (npm audit/pip audit), secrets no codigo-fonte, configuracoes inseguras (debug mode, CORS wildcard), headers de seguranca ausentes, endpoints sem autenticacao que deveriam ter, permissoes excessivas e compliance com LGPD/GDPR. Gere um relatorio HTML com score de seguranca.
```

Output esperado:

- Script de audit automatizado
- Verificação de dependências
- Scan de secrets
- Verificação de configurações
- Relatório HTML com score
- Integração com CI/CD
- Checklist manual para itens não-automatizáveis

Dica pro: Integre este checklist como gate no pipeline CI/CD — nenhum deploy sem score mínimo de 80%.

Capítulo 11 — Performance

10 skills para otimizar velocidade, consumo de recursos e experiência do usuário.

Skill #115 — Analisar e Otimizar Bundle Size

Categoria: Performance | **Nível:** Intermediário

Quando usar: Quando seu bundle JavaScript está grande demais e impactando o carregamento.

Prompt:

Analise o bundle size deste projeto. Identifique: dependências pesadas que podem ser substituídas por alternativas mais leves, código morto (tree-shaking), imports que puxam a biblioteca inteira (lodash vs lodash-es), assets não otimizados e duplicações. Para cada problema, implemente o fix e gere um relatório antes/depois com tamanho em KB de cada chunk. Configure webpack-bundle-analyzer ou similar.

Output esperado:

- Relatório de análise do bundle
- Substituição de dependências pesadas
- Tree-shaking configurado
- Imports otimizados
- Comparativo antes/depois em KB
- Configuração de bundle analyzer

Dica pro: Defina um budget máximo de bundle (ex: 200KB gzipped) e adicione check no CI.

Skill #116 — Gerar Estratégias de Lazy Loading

Categoria: Performance | **Nível:** Intermediário

Quando usar: Quando precisar carregar componentes e módulos sob demanda para melhorar o tempo inicial.

Prompt:

Implemente lazy loading completo neste projeto. Inclua: code splitting por rota (React.lazy + Suspense ou equivalente), lazy loading de imagens com Intersection Observer e placeholder blur, lazy loading de componentes pesados (editores, gráficos, mapas), prefetch de rotas prováveis no hover do link, skeleton screens durante loading e fallback graceful para erros de carregamento.

Output esperado:

- Code splitting por rota

- Lazy loading de imagens com blur
- Componentes pesados carregados sob demanda
- Prefetch de rotas
- Skeleton screens
- Error boundaries para chunks

Dica pro: Use prefetch para rotas que o usuário provavelmente vai acessar (ex: hover em links de navegação).

Skill #117 — Criar Camadas de Cache (HTTP, App, DB)

Categoria: Performance | **Nível:** Avançado

Quando usar: Quando precisar reduzir latência e carga no servidor com caching estrategico.

Prompt:

```
Implemente uma estrategia de caching em 3 camadas para este projeto. Camada 1 (HTTP): configure Cache-Control, ETag e CDN headers por tipo de recurso. Camada 2 (Aplicacao): implemente cache em memoria (node-cache/lru-cache) e Redis para dados frequentes com TTL e invalidacao. Camada 3 (Database): configure query cache, materialized views para queries pesadas. Inclua estrategia de invalidacao (write-through, cache-aside) e metricas de hit/miss rate.
```

Output esperado:

- Headers HTTP de cache por tipo de recurso
- Cache Redis com TTL e invalidacao
- Cache em memoria para dados hot
- Materialized views para queries pesadas
- Métricas de hit/miss rate
- Testes de invalidacao

Dica pro: Cache-aside é o padrão mais seguro — só escreva no cache após ler do banco, nunca o contrario.

Skill #118 — Gerar Pipeline de Otimização de Imagens

Categoria: Performance | **Nível:** Básico

Quando usar: Quando imagens estao pesadas e impactando performance e LCP.

Prompt:

```
Crie um pipeline automatizado de otimizacao de imagens para este projeto. Implemente: conversao automatica para WebP/AVIF com fallback, responsive images com srcset e sizes, compressao com qualidade otimizada (sharp/imagemin), sprites para icones pequenos, lazy loading nativo com aspect-ratio para evitar CLS, CDN de imagens (Cloudinary/imgix) ou processamento local. Integre no build pipeline.
```

Output esperado:

- Conversão automática WebP/AVIF
- Responsive images com srcset
- Compressão otimizada
- Lazy loading com aspect-ratio
- Script de processamento no build
- Configuração de CDN de imagens

Dica pro: AVIF tem 20% melhor compressão que WebP mas demora mais para encodar — use WebP para build e AVIF para CDN.

Skill #119 — Otimizar Queries de Banco de Dados

Categoria: Performance | **Nível:** Intermediário

Quando usar: Quando queries lentas estão impactando o tempo de resposta da API.

Prompt:

Analise todas as queries deste projeto e otimize para performance. Identifique: queries N+1 e resolva com eager loading, queries sem index e sugira indexes, queries que retornam dados demais (SELECT *), joins ineficientes, queries que podem ser cacheadas, transactions longas que bloqueiam. Para cada otimização, mostre o EXPLAIN antes/depois e o impacto esperado em ms.

Output esperado:

- Lista de queries problemáticas
- Resolução de N+1 com eager loading
- Indexes sugeridos e criados
- Queries otimizadas (SELECT específico)
- EXPLAIN antes/depois
- Estimativa de impacto em ms

Dica pro: Habilite slow query log (> 100ms) no banco para monitorar continuamente.

Skill #120 — Gerar Estratégias de CDN

Categoria: Performance | **Nível:** Intermediário

Quando usar: Quando precisar servir assets e conteúdo mais perto dos usuários.

Prompt:

Configure uma estratégia de CDN completa para este projeto. Implemente: setup de CDN (CloudFront/Cloudflare) com origins corretas, cache rules por tipo de asset (imutáveis com hash = 1 ano, HTML = no-cache), compressão Brotli/gzip no edge, purge automático no deploy, custom domain com SSL, page rules para cachear API responses quando aplicável e monitoramento de hit rate.

Output esperado:

- Configuração de CDN (IaC)
- Cache rules por tipo de asset
- Compressão no edge
- Purge automático no deploy
- Custom domain + SSL
- Monitoramento de hit rate

Dica pro: Use content hashing nos nomes de arquivos (app.a1b2c3.js) para cache imutável de 1 ano.

Skill #121 — Criar Code Splitting Avançado

Categoria: Performance | **Nível:** Intermediário

Quando usar: Quando o bundle monolitico esta impactando o carregamento inicial.

Prompt:

```
Implemente code splitting avançado neste projeto. Configure: split por rota automatico, split de vendor chunks (react, lodash separados), chunks compartilhados entre rotas (commons), dynamic imports para features pesadas, preload/prefetch hints para chunks criticos e analise do chunk graph para identificar dependencias circulares. Gere relatorio visual do split com webpack-bundle-analyzer.
```

Output esperado:

- Split por rota automático
- Vendor chunks separados
- Commons chunks compartilhados
- Dynamic imports configurados
- Preload/prefetch hints
- Relatório visual do bundle

Dica pro: O vendor chunk deve ser separado — ele muda menos que seu código e fica cacheado por mais tempo.

Skill #122 — Otimizar Server-Side Rendering

Categoria: Performance | **Nível:** Avançado

Quando usar: Quando seu SSR está lento ou consumindo muitos recursos do servidor.

Prompt:

```
Otimize o Server-Side Rendering deste projeto para performance maxima. Implemente: streaming SSR (renderToPipeableStream/ReadableStream) em vez de renderToString, cache de paginas renderizadas (ISR pattern), component-level caching para partes estaticas, abort timeout para renderizacoes lentas com fallback para client-side, prefetch de dados em paralelo (nao em cascata) e metricas de TTFB por rota.
```

Output esperado:

- Streaming SSR implementado
- Cache de páginas (ISR)
- Component-level caching
- Timeout com fallback
- Data fetching em paralelo
- Métricas de TTFB

Dica pro: Streaming SSR melhora TTFB dramaticamente — o browser começa a processar enquanto o servidor ainda renderiza.

Skill #123 — Configurar Connection Pooling e Keep-Alive

Categoria: Performance | **Nível:** Intermediario

Quando usar: Quando conexões com banco ou serviços externos estão criando overhead.

Prompt:

```
Configure connection pooling otimizado para todas as conexoes externas deste projeto. Implemente: pool de conexoes do banco (min 5, max 20, idle timeout 10s), HTTP keep-alive para APIs externas com agent pool, connection reuse para Redis, retry com exponential backoff para conexoes falhas, circuit breaker para dependencias instaveis e health check periodico das conexoes no pool. Monitore metricas de pool usage.
```

Output esperado:

- Pool de banco configurado e dimensionado
- HTTP keep-alive para APIs externas
- Redis connection reuse
- Retry com exponential backoff
- Circuit breaker implementado
- Métricas de pool usage

Dica pro: Dimensione o pool baseado em $(\text{num_cores} * 2) + 1$ para conexões de banco — mais que isso causa contenção.

Skill #124 — Gerar Profiling e Otimização de Memória

Categoria: Performance | **Nível:** Avancado

Quando usar: Quando sua aplicação está consumindo muita memória ou tem memory leaks.

Prompt:

```
Analise e otimize o consumo de memoria deste projeto. Implemente: identificacao de memory leaks com heap snapshots, eliminacao de closures que retém referencias, limpeza de event listeners e timers no unmount, streams em vez de buffers para arquivos grandes, paginacao de resultados grandes do banco (nao carregar tudo em memoria), weak references para caches e monitoramento de heap usage em producao com alertas.
```

Output esperado:

- Identificacao de memory leaks
- Cleanup de event listeners e timers
- Streams para processamento de arquivos
- Paginação de resultados grandes
- Weak references para caches
- Monitoramento de heap em produção
- Alertas configurados

Dica pro: Use `--max-old-space-size` para limitar a heap do Node.js e forçar a identificacao de leaks mais cedo.

Capítulo 12 — Frontend

10 skills para criar interfaces modernas, acessíveis e performáticas.

Skill #125 — Gerar Layouts Responsivos a Partir de Descrições

Categoria: Frontend | **Nível:** Básico

Quando usar: Quando precisar criar layouts responsivos rapidamente a partir de uma descrição visual.

Prompt:

Crie um layout responsivo completo baseado nesta descrição: [descreva o layout]. Use CSS Grid e Flexbox, mobile-first approach, breakpoints padrão (640px, 768px, 1024px, 1280px), unidades relativas (rem, %, vw/vh), container queries onde aplicável e teste em 5 resoluções. Se estiver usando Tailwind, use classes utilitárias. Garanta que não haja scroll horizontal em nenhuma resolução.

Output esperado:

- Layout responsivo completo
- Mobile-first com breakpoints
- CSS Grid + Flexbox
- Sem scroll horizontal
- Container queries
- Testado em múltiplas resoluções

Dica pro: Descreva o layout com detalhes visuais: "header fixo com logo a esquerda e menu a direita, hero full-width..."

Skill #126 — Criar Design System / Biblioteca de Componentes

Categoria: Frontend | **Nível:** Avançado

Quando usar: Quando precisar padronizar componentes visuais entre projetos ou times.

Prompt:

Crie um design system completo para este projeto. Inclua: tokens de design (cores, tipografia, espaçamento, sombras, border-radius) como CSS custom properties, componentes base (Button, Input, Select, Modal, Toast, Card, Badge, Avatar, Tooltip), variantes (primary, secondary, ghost, destructive), estados (hover, focus, disabled, loading), composição (ButtonGroup, FormField), documentação inline e Storybook stories para cada componente.

Output esperado:

- Design tokens como CSS variables

- 10+ componentes base
- Variantes e estados completos
- Composicao de componentes
- Storybook stories
- Documentação de uso

Dica pro: Defina os tokens primeiro — componentes mudam, mas tokens de cor e espaçamento são a fundacao.

Skill #127 — Gerar Validação de Formularios com Estados de Erro

Categoria: Frontend | **Nível:** Intermediario

Quando usar: Quando precisar de formularios com validação robusta e UX de erro elegante.

Prompt:

```
Implemente validacao de formularios completa neste projeto usando React Hook Form (ou equivalente) + Zod. Inclua: validacao em tempo real (onChange) com debounce, mensagens de erro contextuais e acessiveis (aria-describedby), validacao async (verificar email unico), indicadores visuais de campo valido/invalido, focus automatico no primeiro campo com erro, prevencao de submit duplo e schema de validacao compartilhado entre frontend e backend.
```

Output esperado:

- Formularios com React Hook Form + Zod
- Validação em tempo real com debounce
- Mensagens de erro acessíveis
- Validação async
- Focus automático em erros
- Schema compartilhado front/back

Dica pro: Compartilhe o schema Zod entre frontend e backend para ter uma única fonte de verdade.

Skill #128 — Criar Patterns de State Management

Categoria: Frontend | **Nível:** Intermediario

Quando usar: Quando o estado da aplicação está ficando complexo e precisa de organização.

Prompt:

```
Implemente state management otimizado para este projeto. Use Zustand (ou Redux Toolkit) com: stores separados por dominio (auth, ui, data), selectors granulares para evitar re-renders, middleware de persistencia (localStorage/sessionStorage), devtools configurados, actions tipadas, estado derivado com computed/selectors, patterns de loading/error/success para dados async e hidratacao do servidor (se SSR).
```

Output esperado:

- Stores organizados por dominio

- Selectors granulares
- Persistência configurada
- DevTools integrados
- Actions tipadas
- Loading/error states
- Testes de stores

Dica pro: Zustand é mais leve e simples que Redux. Use Redux Toolkit apenas se precisar de middleware complexo ou time-travel debugging.

Skill #129 — Gerar Animacoes e Transicoes

Categoria: Frontend | **Nível:** Intermediario

Quando usar: Quando precisar adicionar animacoes fluidas e transicoes a interface.

Prompt:

Implemente animacoes e transicoes profissionais neste projeto. Inclua: transicoes de pagina/rota (fade, slide), animacoes de entrada de elementos (scroll reveal), micro-interacoes em botoes e inputs (hover, focus, click feedback), skeleton loading animado, transicoes de lista (adicionar/remover itens), animacao de numeros/contadores e reducao de animacoes para prefers-reduced-motion. Use CSS transitions/animations para simples e Framer Motion para complexas.

Output esperado:

- Transicoes de rota
- Scroll reveal animations
- Micro-interações em botoes/inputs
- Skeleton loading animado
- List transitions
- Suporte a prefers-reduced-motion
- Performance otimizada (transform/opacity only)

Dica pro: Anime apenas transform e opacity — essas propriedades rodam na GPU e não causam layout shift.

Skill #130 — Criar Componentes Acessíveis (ARIA)

Categoria: Frontend | **Nível:** Intermediario

Quando usar: Quando precisar garantir que sua interface é usavel por todos, incluindo tecnologias assistivas.

Prompt:

Faca um audit de acessibilidade completo neste projeto e corrija todos os problemas. Implemente: semantica HTML correta (landmarks, headings hierarquicos), ARIA labels e roles onde necessario, navegacao por teclado completa (Tab, Enter, Escape, Arrow keys), focus management (focus trap em modais, skip links), contraste de cores WCAG AA, textos alternativos em imagens, anuncio de mudancas dinamicas (aria-live) e testes automaticos com axe-core.

Output esperado:

- HTML semântico corrigido
- ARIA labels e roles
- Navegação por teclado
- Focus management em modais
- Contraste WCAG AA
- Textos alternativos
- Testes com axe-core
- Relatório de compliance

Dica pro: Teste com leitor de tela real (NVDA no Windows, VoiceOver no Mac) — ferramentas automáticas pegam apenas 30% dos problemas.

Skill #131 — Gerar SEO Meta Tags e Structured Data

Categoria: Frontend | **Nível:** Básico

Quando usar: Quando precisar otimizar seu site para motores de busca.

Prompt:

Implemente SEO completo neste projeto. Inclua: meta tags essenciais (title, description, canonical, robots), Open Graph para redes sociais (og:title, og:image, og:description), Twitter Cards, JSON-LD structured data (Organization, Product, Article, FAQ, BreadcrumbList conforme aplicavel), sitemap.xml dinamico, robots.txt otimizado, hreflang para multiplos idiomas (se aplicavel) e component/helper reutilizavel para gerar meta tags por pagina.

Output esperado:

- Meta tags essenciais por página
- Open Graph tags
- Twitter Cards
- JSON-LD structured data
- sitemap.xml dinâmico
- robots.txt
- Componente reutilizável de SEO

Dica pro: Use o teste de Rich Results do Google para validar seu structured data antes do deploy.

Skill #132 — Criar Configuração PWA (Progressive Web App)

Categoria: Frontend | **Nível:** Intermediario

Quando usar: Quando precisar que sua web app funcione offline e seja instalavel.

Prompt:

```
Transforme este projeto em uma Progressive Web App completa. Implemente: manifest.json com icons em todos os tamanhos, Service Worker com estrategias de cache (Cache First para assets, Network First para API), offline fallback page, background sync para acoes offline, push notifications (opcional), prompt de instalacao customizado, atualizacao automatica do SW com notificacao ao usuario e splash screen nativa.
```

Output esperado:

- manifest.json completo
- Service Worker com cache strategies
- Offline fallback page
- Background sync
- Prompt de instalação
- Auto-update do SW
- Icons em todos os tamanhos
- Testes offline

Dica pro: Use Workbox — ele abstrai as estratégias de cache do Service Worker e evita bugs comuns.

Skill #133 — Migrar CSS-in-JS para Tailwind

Categoria: Frontend | **Nível:** Intermediario

Quando usar: Quando quiser migrar de styled-components/emotion para Tailwind CSS.

Prompt:

```
Migre este projeto de CSS-in-JS (styled-components/emotion) para Tailwind CSS. Para cada componente: converta os estilos para classes utilitarias, extraia patterns repetidos como @apply em componentes CSS, configure o tailwind.config com os design tokens existentes (cores, fontes, espacamentos), mantenha responsividade e dark mode, remova dependencias de CSS-in-JS apos a migracao e verifique que nenhum estilo foi perdido comparando screenshots.
```

Output esperado:

- Componentes migrados para Tailwind
- tailwind.config com design tokens
- Patterns extraidos com @apply
- Responsividade mantida
- Dark mode preservado
- Dependências de CSS-in-JS removidas
- Comparacao visual antes/depois

Dica pro: Migre componente por componente, não tudo de uma vez. Ambos podem coexistir durante a transição.

Skill #134 — Criar Arquitetura Micro-Frontend

Categoria: Frontend | **Nível:** Avançado

Quando usar: Quando múltiplos times precisam desenvolver e deployar partes do frontend independentemente.

Prompt:

```
Implemente uma arquitetura micro-frontend para este projeto. Configure: Module Federation (webpack) ou Single-SPA como framework base, shell app que orquestra os micro-frontends, comunicação entre MFEs via custom events ou shared state, build e deploy independente por MFE, shared dependencies (React, design system) carregadas uma vez, routing distribuído e fallback quando um MFE está indisponível. Inclua docker-compose para desenvolvimento local.
```

Output esperado:

- Shell app configurado
- Module Federation ou Single-SPA
- Comunicação entre MFEs
- Build independente por MFE
- Shared dependencies otimizadas
- Routing distribuído
- Fallback para MFEs indisponíveis
- Docker-compose para dev local

Dica pro: Micro-frontends adicionam complexidade significativa — use apenas quando há times distintos trabalhando em áreas separadas do produto.

Capítulo 13 — Backend

10 skills para construir sistemas backend robustos e escaláveis.

Skill #135 — Gerar Sistema de Queue/Workers

Categoria: Backend | **Nível:** Intermediário

Quando usar: Quando operações pesadas precisam ser processadas em background.

Prompt:

```
Implemente um sistema de filas e workers para este projeto usando BullMQ (Redis) ou similar. Crie: producer que enfileira jobs com prioridade e delay, worker que processa com concurrency configuravel, retry automatico com exponential backoff (max 3 tentativas), dead letter queue para jobs que falham persistentemente, dashboard de monitoramento (Bull Board), tipagem de job data e results, graceful shutdown do worker e testes de processamento.
```

Output esperado:

- Producer e Worker configurados
- Prioridade e delay suportados
- Retry com exponential backoff
- Dead letter queue
- Dashboard de monitoramento
- Tipagem completa
- Graceful shutdown
- Testes de processamento

Dica pro: Configure concurrency baseado no tipo de job — CPU-bound: 1 por core, I/O-bound: 5-10 por worker.

Skill #136 — Criar Arquitetura Event-Driven

Categoria: Backend | **Nível:** Avançado

Quando usar: Quando serviços precisam reagir a eventos sem acoplamento direto.

Prompt:

```
Implemente uma arquitetura event-driven para este projeto. Crie: EventBus central (in-process para monolito, Redis/RabbitMQ/Kafka para distribuido), eventos tipados com schema (UserCreated, OrderPaid, etc), handlers que reagem a eventos, garantia de entrega (at-least-once), idempotencia nos handlers (processar o mesmo evento 2x e safe), event sourcing para entidades criticas (opcional), logging de todos os eventos publicados e consumer groups para scaling horizontal.
```

Output esperado:

- EventBus implementado
- Eventos tipados com schema
- Handlers registrados
- Garantia at-least-once
- Idempotência nos handlers
- Logging de eventos
- Consumer groups

Dica pro: Comece com EventBus in-process — migrar para mensageria distribuida depois é simples se os eventos já estão tipados.

Skill #137 — Gerar Sistema de Cron Jobs

Categoria: Backend | **Nível:** Intermediario

Quando usar: Quando tarefas precisam rodar em horários agendados.

Prompt:

```
Implemente gerenciamento de cron jobs para este projeto. Use node-cron ou agenda.js e crie: jobs registrados com cron expression e descricao, concurrency control (evitar que o mesmo job rode em paralelo), distribuicao de jobs entre multiplas instancias (leader election), logging de cada execucao (inicio, fim, duracao, sucesso/falha), retry automatico em falha, monitoring com alertas quando um job nao roda no horario esperado e dashboard de status dos jobs.
```

Output esperado:

- Jobs registrados com cron expressions
- Concurrency control
- Leader election para multi-instance
- Logging de execuções
- Retry automático
- Alertas de jobs atrasados
- Dashboard de status

Dica pro: Use leader election para evitar que o mesmo cron job rode em todas as instâncias — apenas uma deve executar.

Skill #138 — Criar Pipeline de Processamento de Arquivos

Categoria: Backend | **Nível:** Intermediario

Quando usar: Quando precisar processar uploads de arquivos (CSV, Excel, imagens, PDFs).

Prompt:

Crie um pipeline de processamento de arquivos para este projeto. Implemente: upload com validacao (tipo, tamanho), processamento em background (queue), etapas de pipeline (parse → validar → transformar → salvar), progresso rastreavel pelo frontend (WebSocket/SSE), tratamento de erro por linha/item (nao falhar o batch inteiro), download do resultado processado, suporte a CSV, Excel (xlsx) e JSON e cleanup de arquivos temporarios.

Output esperado:

- Pipeline de etapas configurável
- Upload com validação
- Processamento em background
- Progresso em tempo real
- Erro por item (não bloqueia batch)
- Download do resultado
- Cleanup de temporários

Dica pro: Use streams para processar arquivos grandes — carregar um CSV de 1GB inteiro na memoria vai crashar o processo.

Skill #139 — Gerar Sistema de Email com Templates

Categoria: Backend | **Nível:** Básico

Quando usar: Quando precisar enviar emails transacionais ou de notificação.

Prompt:

Implemente envio de emails para este projeto. Crie: servico de email com provider abstrato (Resend, SendGrid, AWS SES), templates HTML responsivos com MJML ou React Email, preview de templates em dev (endpoint /emails/preview), variaveis de template tipadas, queue para envio assincrono, rate limiting (respeitar limites do provider), logging de envios e fallback provider (se um falhar, tenta outro). Templates necessarios: welcome, reset-password, notification, invoice.

Output esperado:

- Serviço de email com provider abstrato
- Templates responsivos (MJML/React Email)
- Preview em dev
- Envio assíncrono via queue
- Rate limiting
- Logging de envios
- 4 templates base criados

Dica pro: Use React Email para templates — e JSX que compila para HTML compatível com todos os email clients.

Skill #140 — Criar Integrações de Pagamento

Categoria: Backend | **Nível:** Avançado

Quando usar: Quando precisar integrar pagamentos (Stripe, Mercado Pago, etc).

Prompt:

```
Implemente integracao de pagamento com [Stripe/Mercado Pago] neste projeto. Inclua: checkout session creation, webhook handler para eventos de pagamento (payment_intent.succeeded, charge.failed, subscription.updated), idempotencia em webhooks, subscription management (criar, cancelar, upgrade/downgrade), invoice generation, retry de webhooks falhos, modo sandbox para testes, tipagem completa dos eventos e testes com mock do provider.
```

Output esperado:

- Checkout implementado
- Webhook handler com idempotência
- Subscription management
- Invoice generation
- Modo sandbox
- Tipagem de eventos
- Testes completos

Dica pro: Processe pagamentos APENAS via webhooks, nunca no redirect do checkout — o usuário pode fechar o browser antes do redirect.

Skill #141 — Gerar Implementação de Search Engine

Categoria: Backend | **Nível:** Avançado

Quando usar: Quando precisar de busca full-text avançada no seu produto.

Prompt:

```
Implemente busca full-text neste projeto. Opcao 1 (simples): use tsvector/tsquery do PostgreSQL com GIN index, ranking por relevancia, highlight dos termos encontrados. Opcao 2 (avancada): integre Elasticsearch/Meilisearch com index mapping, sync automatico do banco, busca fuzzy, filtros facetados, autocomplete/suggestions e paginacao de resultados. Para ambas: crie endpoint /search com query, filters, sort e page. Inclua testes de relevancia.
```

Output esperado:

- Search implementado (PG ou Elasticsearch)
- Ranking por relevancia
- Busca fuzzy e autocomplete
- Filtros e ordenação
- Sync automático do banco
- Endpoint /search
- Testes de relevancia

Dica pro: Comece com tsvector do PostgreSQL — e surpreendentemente bom e não requer infra extra. Migre para Elasticsearch quando precisar de fuzzy search ou facets complexos.

Skill #142 — Criar Sistema de Audit Logging

Categoria: Backend | **Nível:** Intermediario

Quando usar: Quando precisar rastrear quem fez o que é quando no sistema.

Prompt:

```
Implemente audit logging completo neste projeto. Registre: quem (user ID, IP, user agent), o que (acao: create, update, delete, login, export), em que (recurso: User#123, Order#456), quando (timestamp ISO 8601), dados anteriores e posteriores (diff para updates). Crie: middleware automatico para todas as operacoes CRUD, tabela de audit imutavel (append-only, sem UPDATE/DELETE), endpoint de consulta com filtros (por user, recurso, acao, periodo) e retencao configuravel.
```

Output esperado:

- Middleware de audit automático
- Tabela imutável append-only
- Diff de dados em updates
- Endpoint de consulta com filtros
- Retenção configurável
- Testes de auditoria

Dica pro: Armazene o diff (antes/depois) de updates — saber que "algo mudou" sem saber "o que mudou" é inútil.

Skill #143 — Gerar Processamento de Jobs em Background

Categoria: Backend | **Nível:** Intermediario

Quando usar: Quando tarefas demoradas não podem bloquear o request do usuário.

Prompt:

```
Implemente processamento de background jobs para este projeto. Crie: job types tipados (SendEmailJob, GenerateReportJob, ProcessUploadJob), scheduler que distribui jobs, worker pool com concurrency configuravel, progresso rastreavel (job.progress(50)), resultado acessivel apos conclusao (GET /jobs/:id/result), timeout configuravel por tipo de job, metricas (jobs/min, duracao media, falhas) e cleanup de jobs antigos.
```

Output esperado:

- Job types tipados
- Scheduler e worker pool
- Progresso rastreavel
- Resultado acessível via API

- Timeout por tipo
- Métricas de processamento
- Cleanup automático

Dica pro: Sempre defina timeout por tipo de job — jobs sem timeout podem ficar stuck e bloquear o worker para sempre.

Skill #144 — Criar Patterns de Deploy Multi-Region

Categoria: Backend | **Nível:** Avançado

Quando usar: Quando sua aplicação precisa servir usuários em diferentes regiões geográficas.

Prompt:

```
Projete e implemente uma estratégia multi-region para este projeto. Inclua: routing baseado em latência (GeoDNS ou load balancer global), replicação de banco cross-region (read replicas), cache regional (Redis por region), sessões que funcionam cross-region (centralizadas ou replicadas), deploy pipeline que atualiza região por região (canary por region), health checks regionais, failover automático entre regiões e estratégia de consistência de dados (eventual vs strong).
```

Output esperado:

- Routing geográfico configurado
- Replicação de banco cross-region
- Cache regional
- Sessões cross-region
- Deploy canary por região
- Failover automático
- Documentação de consistência

Dica pro: Aceite consistência eventual para leitura e direcione escritas para a região primária — simplicidade > perfeição.

Capítulo 14 — AI / ML

8 skills para integrar inteligencia artificial nos seus projetos.

Skill #145 — Gerar Prompt Engineering Patterns

Categoria: AI/ML | **Nível:** Intermediario

Quando usar: Quando precisar criar prompts otimizados para LLMs no seu produto.

Prompt:

Crie uma biblioteca de prompt patterns otimizados para as features de IA deste projeto. Inclua: system prompts com role + context + constraints, chain-of-thought templates para raciocínio complexo, few-shot examples para consistencia de output, output schemas com Zod para structured output, prompt versioning (v1, v2) com A/B testing, templates com variaveis substituiveis e testes automatizados de qualidade de prompt (assertivas sobre output).

Output esperado:

- System prompts otimizados
- Chain-of-thought templates
- Few-shot examples
- Output schemas (Zod)
- Prompt versioning
- Templates parametrizados
- Testes de qualidade

Dica pro: Versione seus prompts como código — mudanças em prompts impactam output tanto quanto mudanças em código.

Skill #146 — Criar Pipeline RAG (Retrieval-Augmented Generation)

Categoria: AI/ML | **Nível:** Avancado

Quando usar: Quando seu produto precisa de IA que responde com base nos seus dados específicos.

Prompt:

Implemente um pipeline RAG completo para este projeto. Inclua: ingestao de documentos (PDF, Markdown, HTML) com chunking inteligente (por secao, nao por tamanho fixo), geracao de embeddings (OpenAI/Cohere), armazenamento em vector database (Pinecone/pgvector/Qdrant), busca semantica com re-ranking, prompt assembly (system + context chunks + user query), streaming de resposta, citacoes com fonte e pagina e avaliacao de qualidade (faithfulness, relevance).

Output esperado:

- Ingestao com chunking inteligente
- Embeddings gerados e armazenados
- Vector database configurado
- Busca semântica com re-ranking
- Prompt assembly otimizado
- Streaming de resposta
- Citacoes com fontes
- Avaliação de qualidade

Dica pro: Chunking por seção semântica (headers, paragrafos) gera resultados muito melhores que chunking por número de tokens.

Skill #147 — Gerar Setup de Embeddings e Vector Search

Categoria: AI/ML | **Nível:** Intermediario

Quando usar: Quando precisar de busca semântica (por significado, não apenas keywords).

Prompt:

Implemente busca por embeddings neste projeto. Configure: geracao de embeddings com OpenAI text-embedding-3-small (ou alternativa), armazenamento em pgvector (extensao do PostgreSQL) ou Pinecone, index HNSW para busca rapida, endpoint de busca semantica com threshold de similaridade, hibrido: combine busca semantica + keyword search (BM25) para melhores resultados, cache de embeddings e batch processing para ingestao inicial.

Output esperado:

- Geração de embeddings configurada
- Vector database com index HNSW
- Endpoint de busca semântica
- Busca hibrida (semântica + keyword)
- Cache de embeddings
- Batch processing para ingestao
- Testes de relevancia

Dica pro: text-embedding-3-small da OpenAI é o melhor custo-beneficio — 5x mais barato que ada-002 com qualidade similar.

Skill #148 — Criar Orquestracao de AI Agents

Categoria: AI/ML | **Nível:** Avancado

Quando usar: Quando precisar de agentes de IA que executam tarefas complexas multi-step.

Prompt:

Implemente um sistema de AI agents para este projeto. Crie: agent base com loop de pensamento (think → act → observe), ferramentas (tools) que o agent pode chamar (buscar no banco, chamar APIs, enviar email), orquestrador que coordena multiplos agents, memoria de curto prazo (contexto da conversacao) e longo prazo (banco de dados), limites de segurança (max iteracoes, timeout, actions permitidas), logging de cada step do agent e interface de chat para interacao.

Output esperado:

- Agent base com loop think-act-observe
- Tools registraveis
- Orquestrador multi-agent
- Memoria curto e longo prazo
- Limites de segurança
- Logging detalhado
- Interface de chat

Dica pro: Sempre defina um limite máximo de iteracoes (10-20) — agents sem limite podem entrar em loop infinito e consumir tokens indefinidamente.

Skill #149 — Gerar Parsing de Structured Output de LLMs

Categoria: AI/ML | **Nível:** Intermediario

Quando usar: Quando precisar de output estruturado (JSON, listas, tabelas) de LLMs.

Prompt:

Implemente structured output parsing para as chamadas de LLM deste projeto. Use: Zod schemas para definir o formato esperado, mode "json" na API da Anthropic/OpenAI, parsing com validacao e retry automatico em caso de formato invalido, fallback para parsing manual (regex) quando o mode json falha, tipagem TypeScript inferida do schema, logging de parse failures para melhorar os prompts e testes com outputs reais capturados.

Output esperado:

- Schemas Zod para cada output
- Parsing com validação
- Retry automático em falha
- Fallback para parsing manual
- Tipagem TypeScript inferida
- Logging de failures
- Testes com outputs reais

Dica pro: Inclua exemplos no prompt do formato JSON esperado — reduz parse failures em 80%+.

Skill #150 — Criar Handlers de Streaming Response

Categoria: AI/ML | **Nível:** Intermediario

Quando usar: Quando precisar exibir respostas de LLM em tempo real (token por token).

Prompt:

Implemente streaming de respostas LLM neste projeto. Backend: use streaming API da Anthropic/OpenAI, transmita via SSE (Server-Sent Events) para o frontend, acumule a resposta completa para salvar no banco, trate erros mid-stream (conexão cortada, rate limit), adicione timeout e cancellation (AbortController). Frontend: renderize tokens incrementalmente, mostre indicador de "pensando", suporte a Markdown rendering progressivo, botão de "parar geração" e scroll automático.

Output esperado:

- Streaming API configurada
- SSE para o frontend
- Acumulação para persistência
- Error handling mid-stream
- Cancellation com AbortController
- Frontend com rendering progressivo
- Botão de parar
- Scroll automático

Dica pro: Use SSE (não WebSocket) para streaming de LLM — e unidirecional, mais simples e funciona com HTTP/2.

Skill #151 — Gerar Features Powered by AI

Categoria: AI/ML | **Nível:** Intermediario

Quando usar: Quando quiser adicionar funcionalidades inteligentes ao seu produto.

Prompt:

Analise este projeto e sugira 5 features que podem ser potencializadas com IA. Para as 3 mais impactantes, implemente: endpoint de API que recebe input do usuário, chamada ao LLM com prompt otimizado, post-processing do resultado, cache de respostas similares para reduzir custos, rate limiting por usuário, fallback para quando a IA está indisponível e métricas de uso e qualidade. Exemplos: resumo automático, categorização, sugestões, tradução, análise de sentimento.

Output esperado:

- 5 features sugeridas com justificativa
- 3 implementadas com API
- Prompts otimizados
- Cache de respostas
- Rate limiting
- Fallback para indisponibilidade
- Métricas de uso

Dica pro: Sempre tenha fallback para quando a IA falha — nenhuma feature deve ser 100% dependente de uma API externa.

Skill #152 — Criar Setup de Avaliação e A/B Testing de Modelos

Categoria: AI/ML | **Nível:** Avançado

Quando usar: Quando precisar comparar modelos, prompts ou configurações de IA.

Prompt:

```
Implemente um sistema de avaliacao e A/B testing para as features de IA deste projeto. Crie: dataset de avaliacao com inputs e outputs esperados, metricas automaticas (accuracy, latencia, custo por request), pipeline de eval que roda automaticamente quando prompts mudam, A/B testing framework (50/50 split entre modelo/prompt A e B), dashboard com resultados comparativos, CI integration (bloquear deploy se qualidade cair) e logging de todas as interacoes para criar novos test cases.
```

Output esperado:

- Dataset de avaliação
- Métricas automáticas
- Pipeline de eval automatizado
- A/B testing framework
- Dashboard comparativo
- CI integration
- Logging para novos test cases

Dica pro: Comece com 20-30 test cases reais de usuários — e mais valioso que 1000 test cases sinteticos.

Capítulo 15 — Project Management

8 skills para organizar, planejar e gerenciar projetos técnicos com eficiência.

Skill #153 — Gerar Estrutura de Projeto a Partir de Requisitos

Categoria: Project Management | **Nível:** Básico

Quando usar: Quando receber requisitos e precisar planejar a estrutura técnica do projeto.

Prompt:

A partir destes requisitos: [cole os requisitos ou user stories]. Gere a estrutura tecnica completa do projeto. Inclua: stack recomendada com justificativa, estrutura de pastas, lista de dependencias, schema de banco de dados inicial, endpoints de API necessarios, componentes de UI principais, estimativa de complexidade por modulo (baixa/media/alta) e ordem recomendada de implementacao (o que construir primeiro). Priorize features por impacto vs esforco.

Output esperado:

- Stack com justificativa
- Estrutura de pastas
- Dependências listadas
- Schema de banco inicial
- Endpoints planejados
- Componentes de UI
- Priorizacao impacto vs esforco
- Roadmap de implementação

Dica pro: Comece pelo "walking skeleton" — fluxo principal funcionando end-to-end antes de features secundarias.

Skill #154 — Criar Documentos de Especificacao Técnica

Categoria: Project Management | **Nível:** Intermediario

Quando usar: Quando uma feature precisa de spec técnica antes da implementação.

Prompt:

Crie uma especificacao tecnica completa para: [descreva a feature]. Inclua: objetivo e escopo (o que esta e o que NAO esta incluso), arquitetura proposta (com diagrama Mermaid), mudancas no banco de dados (migrations necessarias), endpoints de API (request/response), mudancas no frontend (componentes, rotas), dependencias de outros times/servicos, riscos e mitigacoes, plano de rollout (feature flag → beta → GA), metricas de sucesso e estimativa de esforco em dias.

Output esperado:

- Spec completa e estruturada
- Diagrama de arquitetura (Mermaid)
- Mudanças de banco, API e frontend
- Riscos documentados
- Plano de rollout
- Métricas de sucesso
- Estimativa de esforço

Dica pro: A spec deve responder "por que NAO fazer de outro jeito" — alternativas descartadas são tao importantes quanto a solução escolhida.

Skill #155 — Gerar Breakdown de Tasks a Partir de User Stories

Categoria: Project Management | **Nível:** Básico

Quando usar: Quando uma user story precisa ser quebrada em tarefas técnicas estimaveis.

Prompt:

Quebre esta user story em tarefas tecnicas implementaveis: [cole a user story]. Para cada tarefa: titulo descritivo e acionavel, descricao com criterios de aceite, dependencias (o que precisa estar pronto antes), estimativa em story points ou horas, quem deveria fazer (frontend/backend/fullstack), ordem de execucao e se pode ser paralelizada com outra tarefa. Agrupe em: setup, backend, frontend, testes e deploy.

Output esperado:

- Tarefas técnicas granulares
- Critérios de aceite
- Dependências mapeadas
- Estimativas
- Ordem de execução
- Paralelismo identificado

Dica pro: Se uma tarefa tem estimativa > 8h, quebre mais — tarefas grandes escondem complexidade não identificada.

Skill #156 — Criar Recomendações de Sprint Planning

Categoria: Project Management | **Nível:** Intermediario

Quando usar: Quando precisar planejar o próximo sprint baseado no backlog e capacidade.

Prompt:

Analise o estado atual deste projeto (issues abertas, PRs pendentes, dívida técnica) e recomende o planning do próximo sprint. Considere: velocidade do time (baseada nos últimos sprints se disponível), dívida técnica que precisa ser paga, bugs críticos que devem ser priorizados, features alinhadas com o roadmap e capacidade realista (80% da capacidade – 20% para imprevistos). Gere o sprint goal, lista priorizada de itens e riscos do sprint.

Output esperado:

- Sprint goal claro
- Itens priorizados
- Capacidade calculada
- Dívida técnica incluída
- Riscos do sprint
- Buffer para imprevistos

Dica pro: Planeje 80% da capacidade — os 20% restantes serão consumidos por bugs, meetings e contexto switching.

Skill #157 — Gerar Checklists de Code Review

Categoria: Project Management | **Nível:** Básico

Quando usar: Quando o time precisa de um padrão de qualidade para code reviews.

Prompt:

Crie um checklist de code review completo e customizado para este projeto. Categorize por: funcionalidade (código faz o que deve?), segurança (input validado? auth checado? sem SQL injection?), performance (queries otimizadas? sem N+1? cache onde necessário?), legibilidade (nomes claros? funções curtas? sem código morto?), testes (testes adicionados? edge cases cobertos?), arquitetura (segue os padrões do projeto? sem acoplamento excessivo?) e documentação (mudanças documentadas?). Formate como PR template para o GitHub.

Output esperado:

- Checklist por categoria
- PR template para GitHub
- Customizado para a stack do projeto
- Critérios objetivos (não subjetivos)
- Severidade por item (bloqueante/sugestão)

Dica pro: Automatize o que puder (lint, type-check, tests no CI) — o review humano deve focar em lógica e arquitetura.

Skill #158 — Criar Sistema de Tracking de Dívida Técnica

Categoria: Project Management | **Nível:** Intermediário

Quando usar: Quando dívida técnica esta acumulando e precisa ser visibilizada e priorizada.

Prompt:

Analise este projeto e identifique toda a dívida técnica existente. Para cada item: descreva o problema, classifique por tipo (design debt, code debt, test debt, doc debt, infra debt), estime o impacto (alto/medio/baixo), estime o esforço de correcao (horas/dias), identifique o risco de nao corrigir e sugira quando corrigir (proximo sprint, proximo quarter, oportunistico). Gere um arquivo TECH_DEBT.md com o inventario e crie issues no formato do projeto para os top 5.

Output esperado:

- Inventario de dívida técnica
- Classificação por tipo e impacto
- Estimativa de esforço
- Risco de não corrigir
- TECH_DEBT.md gerado
- Top 5 como issues

Dica pro: Aloque 20% de cada sprint para dívida técnica — se deixar acumular, eventualmente a velocidade cai a zero.

Skill #159 — Gerar Estratégias de Atualização de Dependências

Categoria: Project Management | **Nível:** Intermediario

Quando usar: Quando dependências estão desatualizadas e precisam de uma estratégia de update.

Prompt:

Analise as dependencias deste projeto e crie uma estrategia de atualizacao. Identifique: dependencias com vulnerabilidades (npm audit), dependencias mais de 2 major versions atrasadas, dependencias deprecated com alternativas, breaking changes esperados por update. Crie: script que mostra todas as dependencias desatualizadas com changelog links, ordem segura de atualizacao (dependencias internas primeiro), plano de migracao para breaking changes e configure Dependabot/Renovate para updates automaticos com auto-merge para patches.

Output esperado:

- Inventario de dependências desatualizadas
- Vulnerabilidades identificadas
- Ordem segura de atualização
- Plano de migração
- Dependabot/Renovate configurado
- Auto-merge para patches

Dica pro: Configure Renovate com auto-merge para patch updates que passam nos testes — reduz a carga de manutenção em 80%.

Skill #160 — Criar Dashboard de Saude do Projeto

Categoria: Project Management | **Nível:** Avançado

Quando usar: Quando precisar de visibilidade sobre a saude técnica do projeto.

Prompt:

```
Crie um dashboard de saude do projeto que agrega metricas de: qualidade de codigo (cobertura de testes, issues de lint, complexidade ciclomatica), velocidade de entrega (lead time, cycle time, deploy frequency), confiabilidade (uptime, error rate, MTTR), seguranca (vulnerabilidades abertas, tempo medio de correcao) e divida tecnica (% de arquivos sem testes, dependencias desatualizadas). Gere como HTML estatico (atualizavel via CI) ou integre com Grafana. Defina thresholds verde/amarelo/vermelho.
```

Output esperado:

- Dashboard HTML ou Grafana
- Métricas de qualidade, velocidade, confiabilidade
- Thresholds verde/amarelo/vermelho
- Atualização automática via CI
- Tendencias ao longo do tempo
- Alertas quando métricas degradam

Dica pro: Foque em tendencias, não valores absolutos — cobertura caindo de 80% para 70% e mais preocupante que cobertura estavel em 60%.

Capítulo 16 — Code Review

6 skills para revisar código de forma eficiente e construtiva.

Skill #161 — Revisar PR para Bugs e Anti-Patterns

Categoria: Code Review | **Nível:** Intermediário

Quando usar: Quando precisar fazer review de uma Pull Request focando em qualidade.

Prompt:

Revise este PR (ou diff) e identifique: bugs potenciais (null reference, off-by-one, race conditions), anti-patterns (god classes, magic numbers, deep nesting), violações dos patterns do projeto, edge cases não tratados, possibilidade de data loss ou state corruption. Para cada issue: classifique como bloqueante ou sugestão, explique o risco real (não teórico), sugira o fix com exemplo de código. Seja construtivo – comece destacando o que está bom.

Output esperado:

- Bugs potenciais identificados
- Anti-patterns apontados
- Classificação bloqueante/sugestão
- Risco real explicado
- Fixes sugeridos com código
- Pontos positivos destacados

Dica pro: Comece reviews destacando algo positivo — feedback construtivo é mais bem recebido que uma lista de problemas.

Skill #162 — Gerar Comentários de Code Review com Sugestões

Categoria: Code Review | **Nível:** Básico

Quando usar: Quando quiser gerar comentários de review claros e acionáveis.

Prompt:

Analise este código e gere comentários de code review no formato do GitHub. Para cada comentário: indique o arquivo e linha, use prefixos ([Bug], [Suggestion], [Question], [Nit]), explique o problema de forma concisa, sugira a melhoria com bloco suggestion (``suggestion`), adicione link de referência quando relevante (docs, artigos). Foque em impacto real – não comente estilo se há linter, não comente formatação se há Prettier.

Output esperado:

- Comentários com arquivo e linha

- Prefixos claros por tipo
- Sugestões com código
- Links de referência
- Foco em impacto, não estilo

Dica pro: Use o formato suggestion do GitHub — o autor pode aceitar a sugestão com um clique, sem sair do PR.

Skill #163 — Criar Regras de Coding Standards

Categoria: Code Review | **Nível:** Intermediário

Quando usar: Quando o time precisa de regras explícitas de coding standards.

Prompt:

```
Análise os patterns existentes neste projeto e formalize um coding standards document. Inclua: naming conventions (variáveis, funções, arquivos, componentes), estrutura de arquivos e organização de imports, patterns obrigatórios (error handling, logging, tipagem), patterns proibidos (any, console.log em prod, nested callbacks), regras de tamanho (função max 50 linhas, arquivo max 300 linhas, parâmetros max 4). Para cada regra: configure no ESLint/Biome quando possível e adicione ao CLAUDE.md para que o Claude Code siga automaticamente.
```

Output esperado:

- Coding standards documentado
- Naming conventions
- Patterns obrigatórios e proibidos
- Limites de tamanho
- ESLint rules configuradas
- CLAUDE.md atualizado
- Exemplos de bom e mau código

Dica pro: Automatize tudo que puder via linter — regras que dependem de review humano são inconsistentes.

Skill #164 — Revisar para Regressões de Performance

Categoria: Code Review | **Nível:** Avançado

Quando usar: Quando um PR pode impactar performance e precisa de review focado.

Prompt:

```
Revise este PR focando exclusivamente em performance. Verifique: queries N+1 introduzidas, loops com complexidade  $O(n^2)$  ou pior, componentes que vão re-render excessivamente (falta de memo/useMemo/useCallback), chamadas de API desnecessárias ou redundantes, payloads grandes sem paginação, operações síncronas que deveriam ser async, imports pesados que poderiam ser lazy loaded e cache opportunities perdidas. Para cada issue, estime o impacto em ms ou memory.
```

Output esperado:

- Regressões de performance identificadas
- N+1 queries detectadas
- Re-renders excessivos apontados
- Impacto estimado em ms/memory
- Fixes sugeridos
- Benchmark antes/depois

Dica pro: Adicione um "perf budget" no CI — se o bundle cresce mais de 5KB ou latencia sobe mais de 50ms, o PR é bloqueado.

Skill #165 — Gerar Checklists de Security Review

Categoria: Code Review | **Nível:** Intermediario

Quando usar: Quando um PR toca em areas sensiveis (auth, pagamento, dados pessoais).

Prompt:

```
Revise este PR focando em segurança. Verifique: input validation em todos os pontos de entrada, output encoding/escaping, autenticação e autorização corretas (nenhum bypass), dados sensíveis expostos em logs ou responses, SQL injection, XSS, CSRF, SSRF, secrets hardcoded, permissões excessivas, dependências com CVEs conhecidos. Para cada vulnerabilidade: classifique severity (critica/alta/média/baixa), explique o vetor de ataque e sugira o fix.
```

Output esperado:

- Vulnerabilidades identificadas
- Severity classificada
- Vetor de ataque explicado
- Fixes sugeridos
- Nenhum bypass de auth
- Nenhum secret hardcoded

Dica pro: PRs que tocam em autenticao, autorizacao ou pagamento devem ter review de segurança obrigatorio — configure como regra.

Skill #166 — Revisar para Compliance de Acessibilidade

Categoria: Code Review | **Nível:** Intermediario

Quando usar: Quando um PR altera a interface e precisa garantir acessibilidade.

Prompt:

Revise este PR focando em acessibilidade (a11y). Verifique: HTML semântico (uso correto de headings, landmarks, lists), atributos ARIA quando necessários (aria-label, aria-describedby, role), navegação por teclado (todos os interativos focáveis, Tab order lógico, Escape fecha modais), contraste de cores (WCAG AA 4.5:1 para texto, 3:1 para UI), textos alternativos em imagens, estados de focus visíveis e anúncio de mudanças dinâmicas (aria-live). Use axe-core para verificações automáticas.

Output esperado:

- Violações de a11y identificadas
- HTML semântico corrigido
- ARIA atributos adicionados
- Navegação por teclado verificada
- Contraste checado
- axe-core integrado
- Compliance WCAG 2.1 AA

Dica pro: Teste com Tab key — se você não consegue usar o componente sem mouse, usuários de teclado e leitor de tela também não.

Bonus — Workflows Avancados

Como combinar skills, criar seus próprios comandos e automatizar tudo.

1. Como Encadear Múltiplas Skills

O poder real do Claude Code aparece quando você encadeia skills em sequencia. Em vez de executar cada skill isoladamente, crie fluxos completos:

Exemplo: Feature do Zero ao Deploy

```
Passo 1: Use a Skill #153 para gerar a estrutura a partir dos requisitos
Passo 2: Use a Skill #005 para criar as migrations de banco
Passo 3: Use a Skill #002 para scaffoldar a API com CRUD
Passo 4: Use a Skill #074 para adicionar validacao com Zod
Passo 5: Use a Skill #039 para gerar testes unitarios
Passo 6: Use a Skill #041 para gerar testes E2E
Passo 7: Use a Skill #062 para criar o README
Passo 8: Use a Skill #051 para commitar com conventional commits
Passo 9: Use a Skill #052 para gerar a descricao do PR
```

Prompt para encadear:

```
Vou te dar uma sequencia de tarefas. Execute cada uma em ordem, usando o output da anterior como input da proxima. Nao pare entre as etapas – execute tudo de uma vez. Etapa 1: [skill]. Etapa 2: [skill]. Etapa 3: [skill]...
```

2. Configurando CLAUDE.md para Máximo Efeito

O arquivo `CLAUDE.md` na raiz do projeto é a ferramenta mais poderosa que a maioria dos devs ignora. Ele configura o comportamento do Claude Code permanentemente para o seu projeto.

Template Completo:

```
# Instrucoes do Projeto

## Stack
- Runtime: Node.js 20 LTS
- Linguagem: TypeScript 5.4 (strict mode)
- Framework: Next.js 14 (App Router)
- Banco: PostgreSQL 16 + Prisma ORM
- Cache: Redis 7
- Testes: Vitest (unit) + Playwright (E2E)
- Estilo: Tailwind CSS 3
- Lint: ESLint + Prettier (ja configurados)

## Convencoes de Codigo
- Funcoes: camelCase, max 50 linhas
- Componentes: PascalCase, um por arquivo
- Arquivos: kebab-case (user-profile.tsx)
- Imports: absolutos (@/components/...), agrupados por tipo
- Error handling: AppError customizado, nunca throw generico
- Logging: structured JSON, nunca console.log em prod

## Patterns Obrigatorios
- Validacao de input com Zod em todo endpoint
- Testes para toda funcao publica (min 80% coverage)
- Conventional Commits (feat:, fix:, refactor:)
- Error boundaries em componentes criticos
- Tipagem explicita em funcoes exportadas (nao depender de inferencia)

## Patterns Proibidos
- any (use unknown + type guard)
- @ts-ignore (corrija o tipo)
- console.log (use o logger)
- Dependencias sem tipos (@types/*)
- Nested ternaries (use early return)
- Magic numbers (use constantes nomeadas)

## Estrutura de Pastas
src/
  app/           # Next.js App Router pages
  components/    # React components (atoms/, molecules/, organisms/)
  lib/           # Business logic, services
  server/        # API routes, middleware
  db/            # Prisma schema, migrations, seeds
  types/         # TypeScript types e interfaces
  utils/         # Pure utility functions
  hooks/         # React custom hooks

## Banco de Dados
- Schema em prisma/schema.prisma
- Migrations: npx prisma migrate dev
- Seed: npx prisma db seed
- Todas as tabelas tem createdAt e updatedAt
- Soft delete com deletedAt onde aplicavel

## Testes
- Unitarios: src/**/*.test.ts
- E2E: e2e/**/*.spec.ts
- Factories em test/factories/
- Rodar: npm test (unit), npm test:e2e (E2E)

## Deploy
- Main branch → producao automatica (Vercel)
- PRs → preview deployment
- Variaveis de ambiente em .env.local (dev) e Vercel dashboard (prod)
```

Dicas para CLAUDE.md:

- Seja específico sobre patterns proibidos — o Claude Code segue a risca
- Inclua a estrutura de pastas — o Claude vai respeitar a organização
- Atualize quando o projeto evoluir — CLAUDE.md desatualizado causa confusão

3. Criando Slash Commands Personalizados

Você pode criar seus próprios comandos que aparecem como `/seu-comando` no Claude Code.

Como criar:

1. Crie a pasta `.claude/commands/` no seu projeto
2. Crie um arquivo Markdown para cada comando (ex: `deploy-check.md`)
3. O nome do arquivo vira o nome do comando (`/deploy-check`)

Exemplo: `/deploy-check`

Crie `.claude/commands/deploy-check.md`:

```
Faca as seguintes verificacoes antes do deploy:

1. Rode `pnpm lint` e reporte erros
2. Rode `pnpm type-check` e reporte erros
3. Rode `pnpm test` e reporte falhas
4. Verifique se todas as variaveis de ambiente em .env.example tem valor em .env
5. Verifique se ha migrations pendentes
6. Rode `pnpm build` e reporte erros
7. Verifique se o ultimo commit tem mensagem conventional

Se tudo passar, diga "PRONTO PARA DEPLOY" em verde.
Se algo falhar, liste os problemas e sugira fixes.
```

Exemplo: `/review`

Crie `.claude/commands/review.md`:

```
Faca um code review completo das mudancas nesta branch (git diff main..HEAD).

Verifique:
- [ ] Bugs potenciais
- [ ] Seguranca (input validation, auth, XSS)
- [ ] Performance (N+1, re-renders, bundle size)
- [ ] Testes (cobertura adequada, edge cases)
- [ ] Tipos (sem any, tipagem correta)
- [ ] Convencoes (segue CLAUDE.md)

Para cada problema, indique: arquivo:linha, severidade e fix sugerido.
```

Exemplo: `/new-feature`

Crie `.claude/commands/new-feature.md` :

Ajude-me a implementar uma nova feature.

1. Pergunte o nome e descricao da feature
2. Crie a branch (feature/nome-da-feature)
3. Gere a spec tecnica (Skill #154)
4. Crie migrations se necessario (Skill #005)
5. Implemente a API (Skill #002)
6. Adicione validacao (Skill #074)
7. Gere testes (Skill #039)
8. Commite com conventional commits (Skill #051)

Execute cada passo e mostre progresso.

4. Patterns de Multi-Agentes

Para projetos complexos, você pode configurar o Claude Code para operar como múltiplos agentes especializados.

Como funciona:

No seu CLAUDE.md, defina papeis:

```
## Agentes Disponiveis

Quando eu disser "modo [agente]", assuma o papel correspondente:

### /modo arquiteto
- Foque em decisoes de arquitetura e design
- Nao escreva codigo – desenhe diagramas e specs
- Questione requisitos e identifique riscos

### /modo implementador
- Foque em escrever codigo limpo e testado
- Siga a spec existente a risca
- Nao questione arquitetura – implemente

### /modo reviewer
- Foque em encontrar bugs, security issues e anti-patterns
- Seja critico mas construtivo
- Sugira fixes com codigo

### /modo debugger
- Foque em diagnosticar e corrigir bugs
- Leia logs, stack traces e reproduza o problema
- Aplique fix minimo e adicione teste de regressao
```

5. Hooks e Automação

O Claude Code suporta hooks que executam comandos automaticamente em resposta a eventos.

Configuração em `.claude/settings.json` :

```
{
  "hooks": {
    "preToolUse": [
      {
        "matcher": "Write",
        "command": "echo 'Escrevendo arquivo...'"
      }
    ],
    "postToolUse": [
      {
        "matcher": "Write",
        "command": "npx prettier --write $CLAUDE_FILE_PATH"
      }
    ]
  }
}
```

Casos de uso:

- **Pos-write:** Rodar Prettier/ESLint automaticamente em arquivos criados
- **Pre-commit:** Verificar que testes passam antes de commitar
- **Pos-edit:** Rodar type-check no arquivo editado

6. Criando suas Próprias Skills

Skills são extensões do Claude Code que adicionam capacidades especializadas.

Estrutura de uma skill:

```
.claude/skills/
minha-skill/
  SKILL.md      # Definição da skill (obrigatorio)
  scripts/      # Scripts auxiliares (opcional)
  data/         # Dados de referencia (opcional)
```

SKILL.md template:

```
---
name: minha-skill
description: Descricao clara do que a skill faz e QUANDO deve ser ativada. Use palavras-chave que o usuario diria naturalmente.
---

# Minha Skill

## O que faz
[Descricao detalhada]

## Como usar
`/minha-skill [argumentos]`

## Workflow
1. [Passo 1]
2. [Passo 2]
3. [Passo 3]

## Regras
- [Regra 1]
- [Regra 2]
```

Dicas para skills:

- A description no frontmatter é o que ativa a skill — use as palavras que o usuário diria
- Mantenha o SKILL.md abaixo de 500 linhas
- Use scripts auxiliares para lógica complexa
- Teste a skill com vários phrasings diferentes

7. Exemplo Completo: Feature do Zero ao Deploy

Aqui esta um workflow completo que combina tudo que você aprendeu:

```
# 1. Inicie o Claude Code no seu projeto
$ cd meu-projeto
$ claude

# 2. Use seu slash command customizado
> /new-feature

# 3. O Claude vai:
#   - Perguntar sobre a feature
#   - Criar a branch
#   - Gerar spec tecnica
#   - Criar migrations
#   - Implementar API + frontend
#   - Adicionar validacao
#   - Gerar testes
#   - Commitar

# 4. Use seu comando de review
> /review

# 5. O Claude revisa o proprio codigo e sugere melhorias

# 6. Use seu comando de deploy check
> /deploy-check

# 7. O Claude verifica que tudo esta pronto

# 8. Crie o PR
> Crie um PR para main com descricao completa

# 9. Done. Feature completa em minutos, nao dias.
```

Parabens por chegar até aqui! Você agora tem 166 skills + os workflows avancados para extrair o máximo do Claude Code. A diferença entre um dev que usa IA é um dev que DOMINA IA esta na sistematizacao — e exatamente isso que este ebook te deu.

Bom coding! 🚀

— WeStack